

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 16: **UE5 Tutorial #02** ***Global Game Logic and Behavioural Logic***

Tutorial Overview

In today's tutorial we will be looking at the individual aspect of the coursework assignment, which tasks you with refactoring an existing UE5 solution written in Blueprints.

This tutorial builds upon the work that we completed in **Tutorial 08 (UE5 Tutorial 01)**, where we created a series of dynamic targets, and added state and manager classes to our game mode.

You will need to find your solution to Tutorial 08 to use as the starting point for today's tutorial.

We want to continue this work by looking at different ways of controlling the behaviour of our target, different ways of dealing with the dynamic spawning of objects and creating some gameplay logic to allow us to record the results of our gameplay interactions as both global and player-level game state.

- Therefore, in **Task 1** we will be adding support to our controlled character and targets so that we can tally up when we have hit the target – we will then report these findings to our Player State and Game State classes.
- In **Task 2**, we will be looking at the interactive elements – in particular, the control unit that is provided in the template – we will be duplicating and modifying this to provide a different gameplay experience with this unit.
- In **Task 3**, we will be looking at Timelines and how they can be used to provide better control over dynamic movements, compared to the Transform Mathematics we used in UE5 Tutorial 01 (*Tutorial 08*).

Task 1: Manipulating State via Actor Logic and Interactions

This Tutorial is using the completed Solution to Tutorial 08 as the basis – if you have not attempted Tutorial 08 first, you **MUST** do so before starting this tutorial.

During Tutorial 08, you were asked to save a copy of your Tutorial 08 Solution code to a backup location such as OneDrive.

1. Locate your solution to Tutorial 08 and extract this to a suitable location;

OR alternatively, if you cannot find your solution, then download the Session 08 Solution code from the module canvas site, under today's tutorial section (**Session 16**) and extract that to a suitable location – this will **ONLY** be available after directed study week.

NOTE: This shouldn't be used as a shortcut – there are lots of important concepts covered in Tutorial 08 and skipping it will just make things harder for you as we progress.

2. Rename the outer folder of the Unreal Project to Tutorial16.
3. Open the UE5 Project using the .uproject. Run the game, and you should have our two types of targets we implemented at the end of Tutorial 08 – **MovingTarget** and **RotatingTarget**.



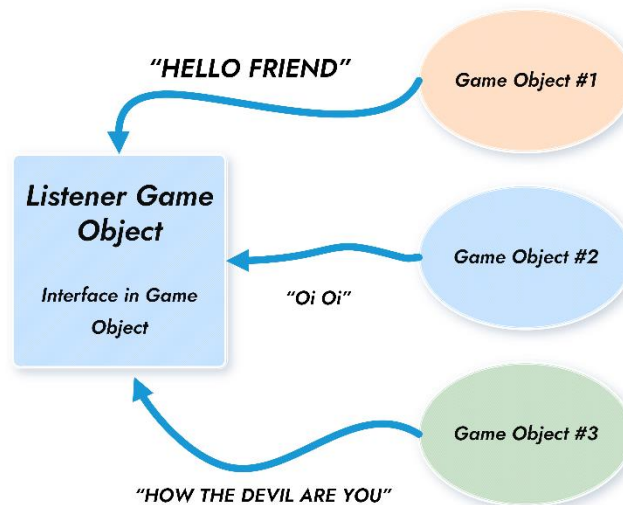
Step 1: Creating an Event Listener in our Character for Hits.

In our first task, we will look at how we can use the events of our interactions with the Targets to change the state of our game – we will do so via the concept of **Event Listeners via Event Dispatchers**.

Event Dispatchers can help us receive messages from other game objects and the only thing they need to be aware of is the Event we are sending a message to.

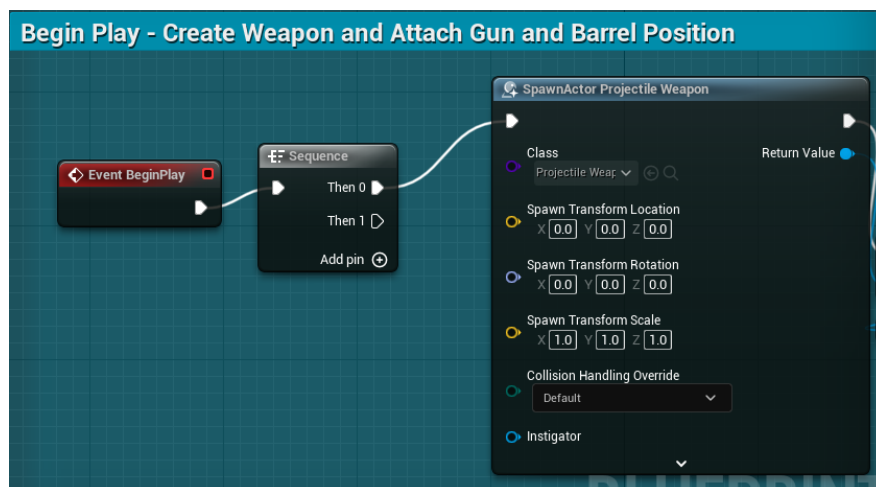
There are several ways we can use Event Dispatchers – all of them based around the design patterns of the **Observer** pattern and **Publish-Subscribe** pattern, depending on how you use them.

We will implement a style of Dispatcher known as an 'Open Ear' dispatcher in UE5 Terms – this is where rather than broadcasting messages, we will register to hear any messages from a particular set of objects.

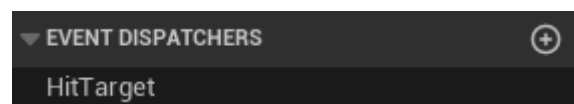


We'll make our **ProjectileCharacter**, be the *listener* object for this event.

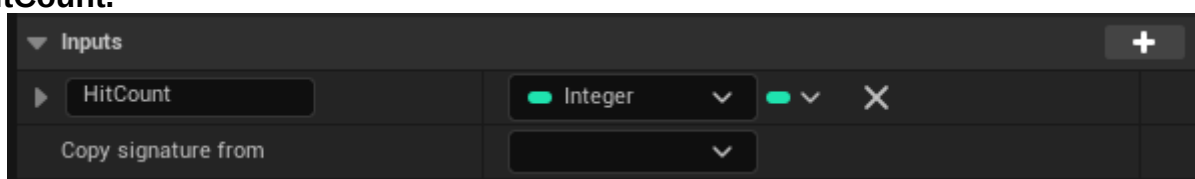
1. Open the **ProjectileCharacter** blueprint and then locate the **Event BeginPlay**.
2. Add a Sequence Node Between the First Node and the Event – Wire up the previous code to **Then 0**



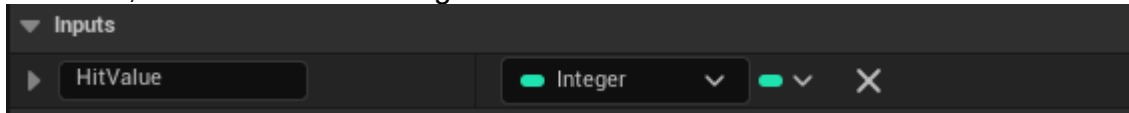
3. You should see that an Event Dispatcher at the bottom of the Blueprint is already present called **HitTarget**.



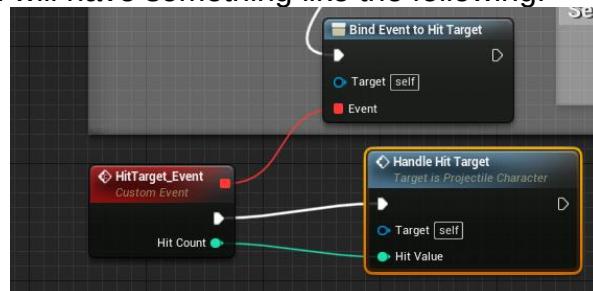
4. In the Details View, have a look at the parameters – this function takes an Integer called **HitCount**.



5. Drag the HitTarget event dispatcher near our Sequence Node choose 'Assign' from the options – this will allow us to associate Dispatcher with a given type of Event. This should create an Event, which has the same number of inputs as we specified in our Dispatcher.
6. Connect **Then 1** to the Bind Event to **Hit Target** node.
7. Add a new function to the Blueprint, called **HandleHitTarget**. To this, add an Input called HitValue, which is also an Integer.



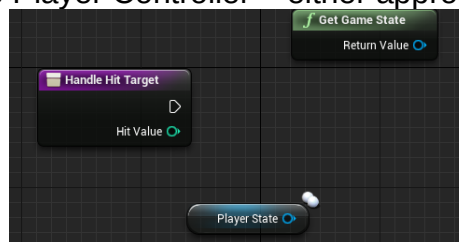
8. Wire up the **HitTarget_Event** that was generated so that HitCount is passed as the HitValue. By now, you will have something like the following:



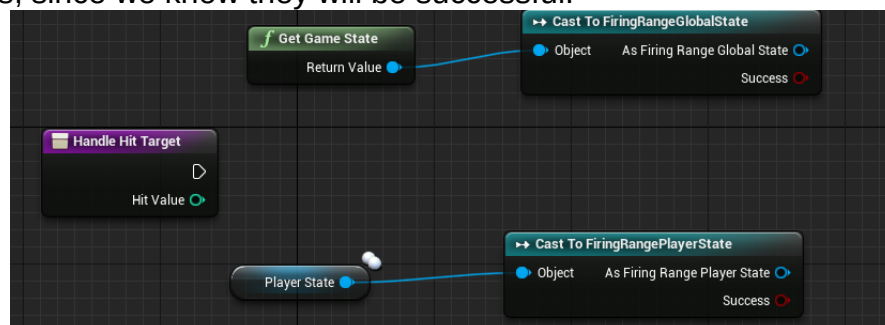
We have bound our new event to our Dispatcher – this is done once when the game starts (BeginPlay is called) and then we call our Handle Hit Target event to handle the event.

Next, we are going to add some logic to our Custom Function – this is going to work with the Game State classes that we have associated with our game mode.

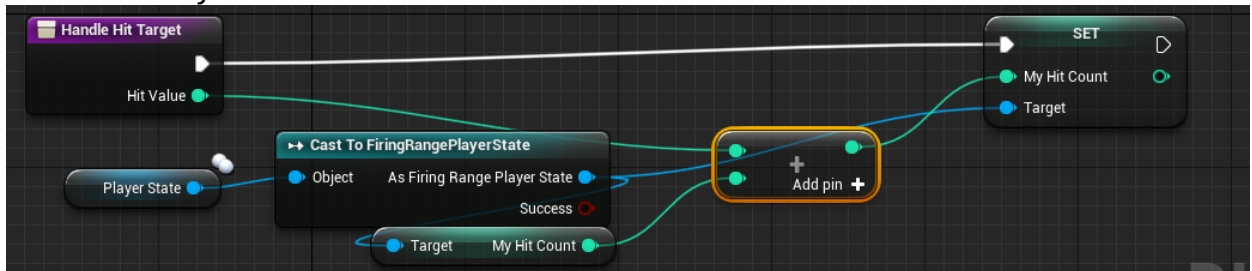
9. Open HandleHitTarget and add nodes which get the game state and get player state – because we are inside a character, as long as our character is possessed, this is the same as getting the state from the Player Controller – either approach is fine.



10. Cast the Game State to our **FiringRangeGlobalState** and the PlayerState to **FiringRangePlayerState** – as we did in Tutorial 08. You should also convert these casts to Pure Casts, since we know they will be successful.



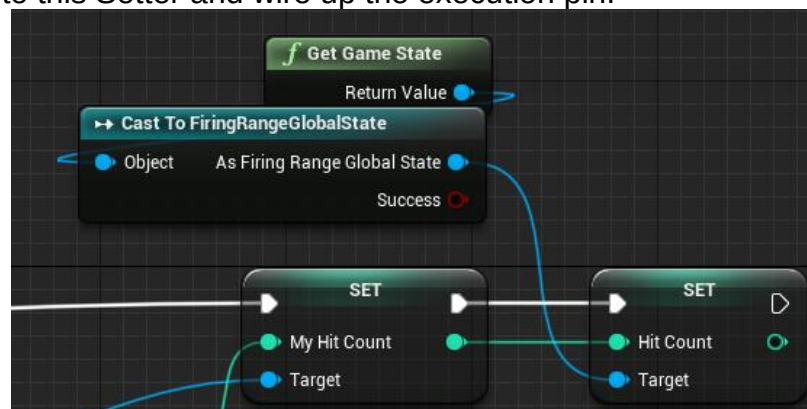
11. Using the **As...** node of our Player State, Get the MyHitCount node and a Set MyHitCount Node.
12. Add our Hit Value Parameter to the MyHitCount getter using an int + int node and then set the value of My Hit count to this value:



NOTE: When we are using an object like state to get a property it is common to stack the nodes on top of each other, to save space, but also make it clear where we are getting the values from – this is particularly useful if we were getting several variables from the PlayerState at once.

13. Because this a single player game, we are going to simply mirror this value into our global game state.

To do this, retrieve the HitCount from the Global State as a Setter and then assign the My HitCount value to this Setter and wire up the execution pin.



If we were multiplayer (e.g. co-op), we could do something more complex to keep a tally of all players' hits. We could use functions in the state classes to allow for complex logic, rather than relying on accessing the variables and modifying them.

14. Compile and Save our Blueprint and close the Blueprints Editor.
15. Repeat this Process with our **RayCastCharacter** – the placement within Begin Play of the Assignment to the Event should be very similar.

So, we have created a dispatcher to receive when a target is hit which will fire an event which calls our function that will add the hit number to the hit count of both the player state and the global game state.

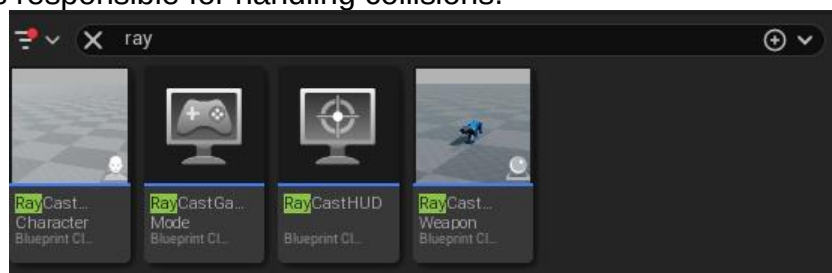
Step 2: Modifying the Target to support Hits

In our next task, we are going to Modify our Paper Target and Dynamic Target Blueprints to support Hit Detection – we know we can already collide with a target – which just want to record when this happens.

1. Open **RailPaperTarget_BP** and in the Event Graph, Right click, type 'Custom' and then choose Add Custom Event... - this will add in a new event to this blueprint. Name the event **OnTargetHit**.
2. To the Firing Range variable list, add a new variable called **NoHits**, which is an Integer. Compile and make sure the default value is set to 0.
3. This time, our event is not going to take any parameter, we are going to just use the event to tell us that the object that a hit has occurred.
4. Wire up **OnTargetHit** to increment the values of NoHits. To do this, drag the NoHits variable as a getter and drag a pin from it a type ++. You should find a function called **Increment Int**. Wire this up to the event. Compile, Save and close the Blueprint.

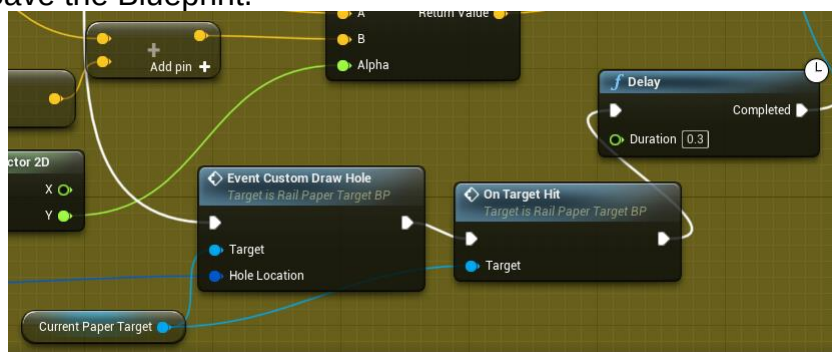


As we currently have two types of character, a RayCast and Projectile, we need to handle both. In both Character's cases, they each have an associated Weapon class – it is the Weapon that is responsible for handling collisions.

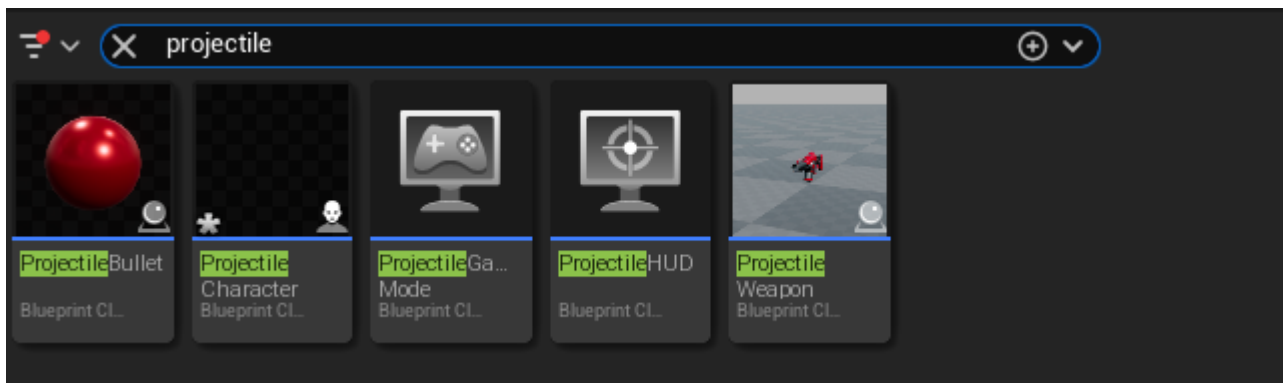


5. Open **RayCastWeapon** and find the function **TargetResponseToBullet**. This is the Ray collision code for Bullets – it handles a Traced Hit and if the Actor we have hit is a Target, we store this target as a temporary variable called CurrentPaperTarget.

Find the code where **Event CustomDrawHole** is called and modify it so that **OnTargetHit** is called after this occurs. Compile and Save the Blueprint.

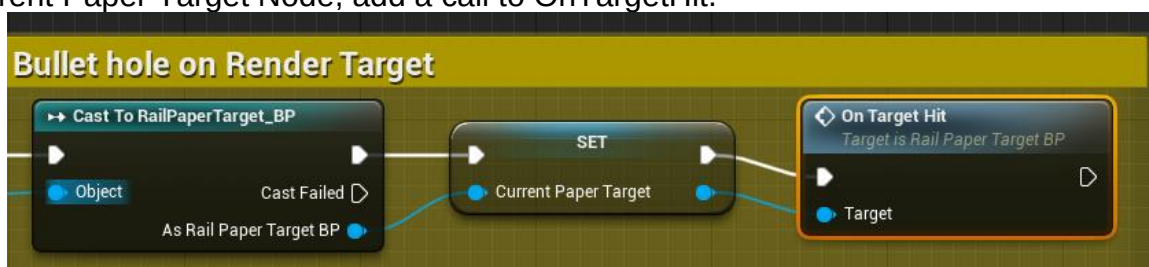


6. Next, we will handle our Projectile Weapon – this weapon spawns a second object, which is the bullet we are firing – it is the bullet which handles the collision.



Open **ProjectileBullet** – this represents the Bullets fired by the ProjectileCharacter.

- Find the Macro **TargetResponseToBullet** and locate the Sequence branch **Then 0**. This again determines if it is a Paper target that has been hit and stores it. After the Set Current Paper Target Node, add a call to OnTargetHit.



This code will only work if the Object is our Target, so it works as a way of filtering our collisions – we only want to register a hit if was a Target that was struck. **Compile and Save our Blueprints.**

- Next, re-open **RailPaperTarget_BP** and put a breakpoint on our Increment action:



We are going to test that our Firing Logic is working.

Run the game and test with our ProjectileCharacter's Weapon – once you are confident projectiles are registering hits and the NoHits is incremented (*hover over it to see the value*), use the World Settings, as shown in Tutorial 08, to change the Default Pawn to our RayCast character.

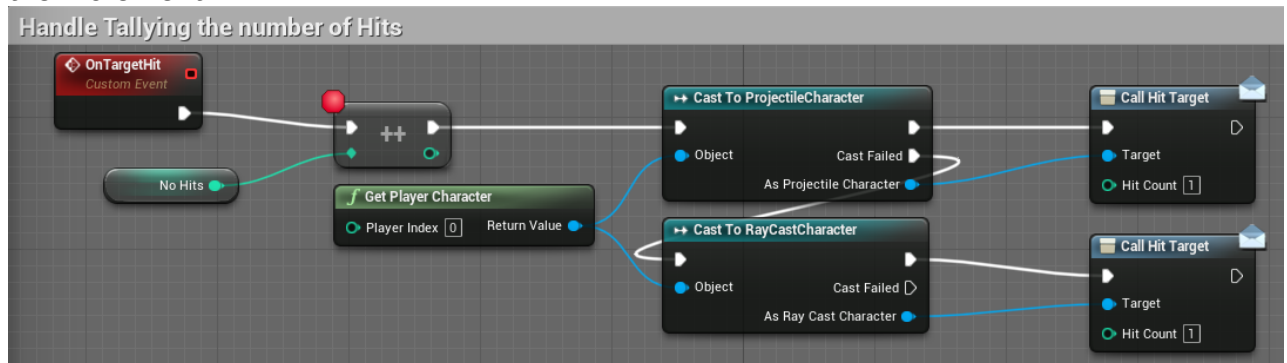
Check that this breakpoint is being struck when we hit the object with this character's weapon as well.

OK, so we can detect hits between our Weapon and the Target – now we just need inform our character that a hit has occurred so it can register it with the Target.

Dispatchers are a great way of passing complex interactions between different blueprints.

Step 3: Inform the Character object of the Hit.

1. Go back to our **OnTargetHit** function inside the Target and add the following nodes after the increment.



We get the current player and cast to each Character – if the cast succeeds, then this then calls our Event Listener (Dispatcher). If the cast fails, we try the same thing on the ray cast character.

HINT: to find the dispatcher node easily – type in HitTarget – make sure it is the dispatcher and not the events/functions in the character (since these could be different for each character).

We could call our Events directly, but by using Event Dispatchers it means we can change which Event/Function is called without needing to modify this Blueprint – this is a process known as **Delegation**. We can stop listening to these events as well by unbinding the events etc, which we could not do with a fixed function call.

Hopefully, you are looking at this code thinking that is not very clean/scalable.

A Target is now couple to both the RayCast and Projectile Character and the downcast checking is ugly – if we added a third type of weapon, then there would be three characters.

Well – this is the point of the assignment – instead, we want a single character (Refactor Character), which can handle all weapon types and in a way where we do not need to know the exact object – you will be doing this when you refactor the character and altering this code should be pretty straightforward.

Step 4: Test the entire process.

1. Open **ProjectileCharacter** and put a break point on HitTarget_Event.
2. Make sure you change the FiringRangeGameMode's Default Pawn
3. Run the game, hit a target, and see if this breakpoint is reached.
4. Repeat the test with the **RayCastCharacter**.

So, we have extended our code with the functionality so that the Target knows how many hits it has received, that different colliding objects (a weapon and a bullet) can communicate with the target and both our player state and game state know this as well by passing the information to the character first. Event Dispatchers are one crucial way of passing changes in state onto other objects.

Task 2: Controlling Actors via Interactive Elements

In this second task, [we are going to be working with our Control Unit](#). Now, our control unit is used as an example of triggering our targets to move, but it could be used in many other ways.

This time, we won't use inheritance, instead we will use **Duplication** to create a separate, unrelated unit that is altered from the original (*Think of Duplication as creating a brother/sister class rather than a Child – this new class will have the same Parent class as the Source class it was duplicated from*) – instead, this control unit will Dynamically Spawn some obstacles into our Firing Range.

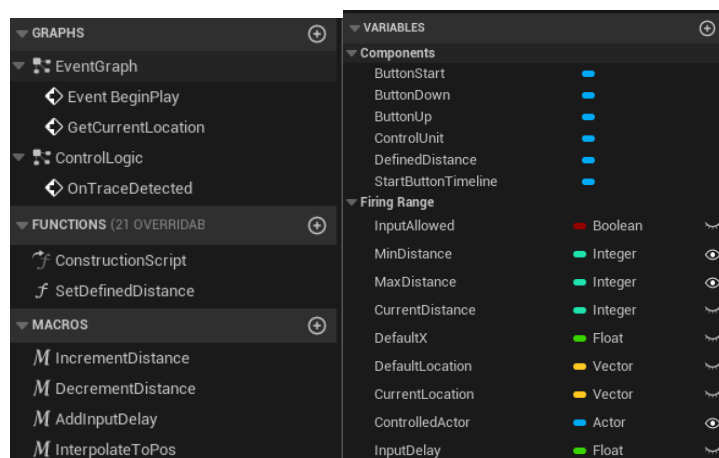
Step 1: Understanding the Existing Control Unit

Make sure you are comfortable with controlling the Control Unit.

1. Run the game and focus your target on each of the buttons of the control unit – the Up and Down arrows on the Unit control the distance of the Target, when the Red Button is pressed, we then trigger the Target to move to this position. To press any of these arrows/buttons, press the **Enter** key whilst hovered over them – we are using Ray Casting to support this. **NOTE:** The Reticule is not perfect (this is something for you to fix!) – you may need to aim above/below to get it to trigger – if you are struggling, find the code for the ray cast in the Character Blueprint and turn on Debug Drawing on the Casting to visualise the ray.

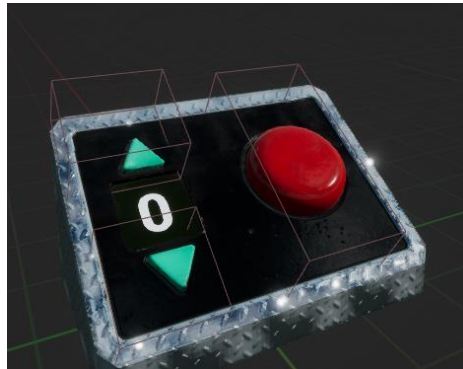


2. Find the Blueprint for **RailControlUnit_BP** in the Content Browser and Open it in the Blueprints Editor.



This is a relatively complex Blueprint – we have quite a few components and variables, and a couple of custom events/functions and some Macros. However, its functionality is relatively simple.

3. Start by looking at the object in the Viewport – you will see how the Hierarchy of Scene Components are being used to construct this unit.



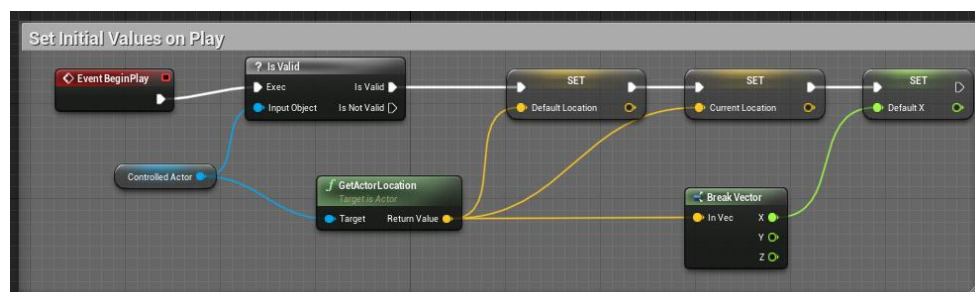
We have a single mesh representing the entire unit, which we have placed three collision boxes over the buttons – the names of which should be self-explanatory. These are used with the Ray Casting to check when a Ray shot in the direction the player is looking from the centre of the screen (our crosshair) for an intersection. We have a TextRenderComponent which is displaying the distance we want to the set the target at (**DefinedDistance**) – the Up and Down buttons dynamically change a variable, which is outputted to this component.

Three of our properties are **instance editable** (modifiable outside the Blueprint/Public):

- **ControlledActor** – The actor whose position we want to change – this is kept as an Actor and not a TargetFollower to make the class flexible – we could actually control any type of object – we can do this because all the functions and data we want to access are common to the Actor class. In the Details view in the Editor, we have linked this control unit to the TargetFollower Instance on the Middle Rail.
- **MinDistance** – the minimum distance from the starting point the target can be moved to.
- **MaxDistance** – the maximum distance from the starting point the target can be moved to.

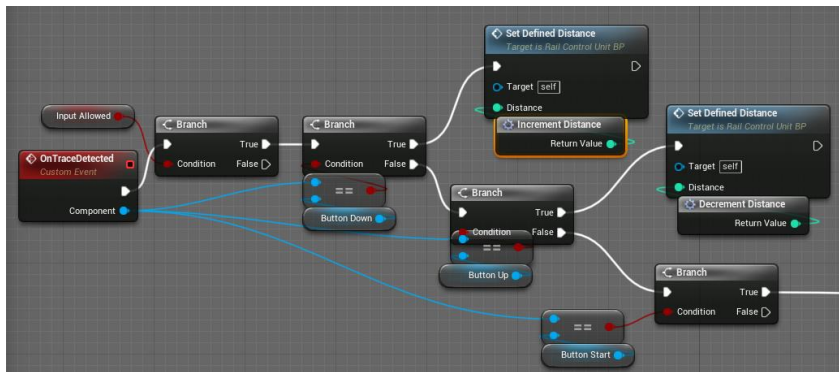
The rest are all variables used in our calculations – we have the Default and Current Location of the object, its starting X position (since we only want backwards and forwards motion), and two variables which control the Input of the buttons – every time we press a button, there is a slight delay until we are allowed to press it again. The code for this is in the **AddInputDelay** macro.

4. Most the Logic is in BeginPlay and **OnTraceDetected**. Let's start with **BeginPlay**:



We check to see if the **ControlledActor** is set via the Is Valid functions (which is quicker than using a branch with the Is Valid property) and use its location to set both the default Location, Current Target Location and the Default X position.

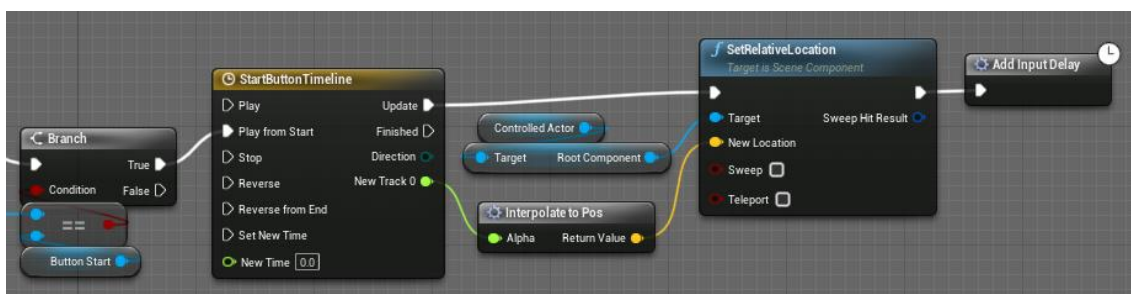
Next, Open **OnTraceDetected**. This is an event, this time called directly rather than using a Dispatcher. **It is the character that oversees handling interactions with this unit** – it is performing the RayCast – when it detects a collision with a control unit, it calls this function, passing over the HitResult object – this contains all the collision data we need.



We have quite a few branches here, but it's straightforward – this is the same as writing:

```
if(inputallowed)
{
    if(button == down)
    ...
    elseif(button == up)
    ...
    elseif(button == start)
    ...
}
```

When the Down button is hit, we decrement the Target distance by 1, when the Up button is hit, we increment it by 1. These values are clamped within the range defined by Min and Max Distance – we use two Macros to help us – this makes the code easier to understand, but also they are re-usable – we might want to add for example some input key presses that do this – these macros could then be used for that.

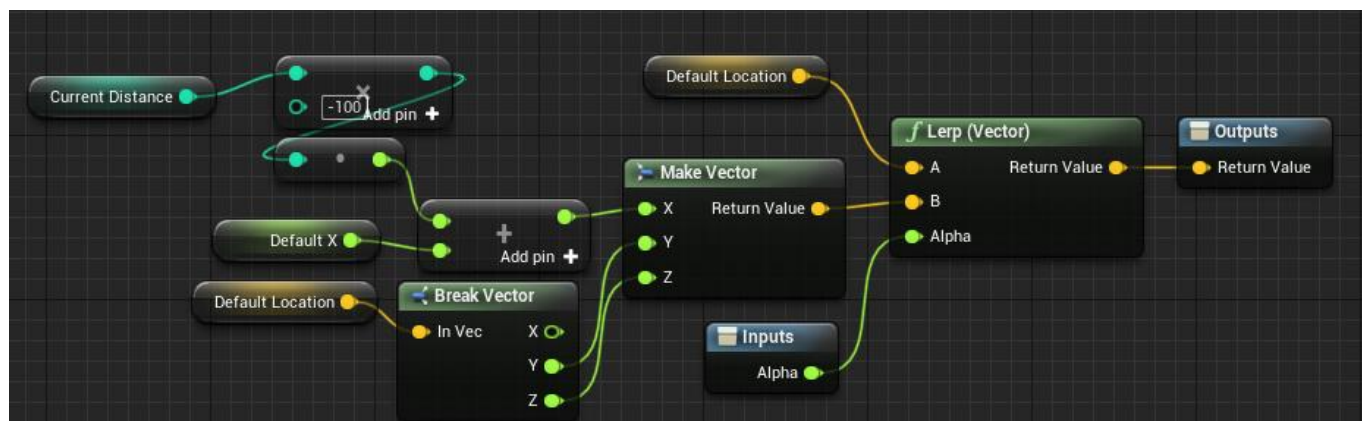


The start button code makes use of a feature known as Timelines – we will look at them in the later tasks of this tutorial.

We use a timeline to control our animation of the movement. Once the button is pressed, the timeline starts and will then be called in each update until it finishes.

This uses a macro which takes the current value from the timeline and Interpolates between the starting position and the target position – it is only controlling the X axis.

The result of this interpolation is set as the position of the Target's Root Component and we then add an input delay – again, Macros are used to make this cleaner.



The main code is the interpolation – again, this is the type of code that quickly gets messy in UE5 and is often easier in C++.

We work out the Target X by taking the Target Distance and scaling it by 100 (so that our control unit can work in single units - e.g. a value of 7 is -700 because we are moving backwards) and adding it to the original X position of our PaperTarget.

We want to keep the Y and Z values of the Target the same as their original (sideways, up/down), so we copy them from our Default Location.

We then use the **Linear Interpolation function (LERP)** to work out an intermediate point between our Original Location (*in your code this is CurrentLocation, to allow for the Starting Point to be adjusted*) and the Target location. Alpha is a value between 0 and 1, where 0 is the start, and 1 is the end, so if we passed it a value of 0.25, we would be $\frac{1}{4}$ of the way away from our starting point, a value 0.5 being $\frac{1}{2}$ way, and a value of 1 meaning we are at the target location.

LERP is an integral concept in Animation:

<https://www.febucci.com/2018/08/easing-functions/> has a good summary – and shows how it is used in an alternative game engine, Unity.

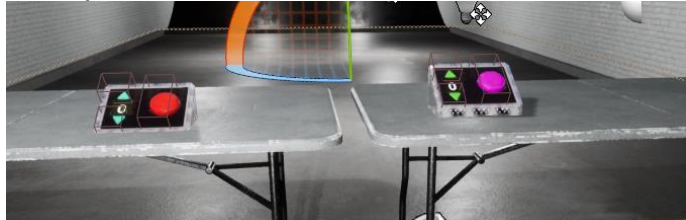
This is the alternative way of writing the logic that we implemented in our **MovingTarget** in Tutorial 08 – mathematically, they end up resolving to the exact same thing, but LERP is often easier to use.

5. Experiment with the ControlUnit by playing the game to make sure you are comfortable with what it is doing.

Step 2: Replacing our Existing Control Unit via Duplication

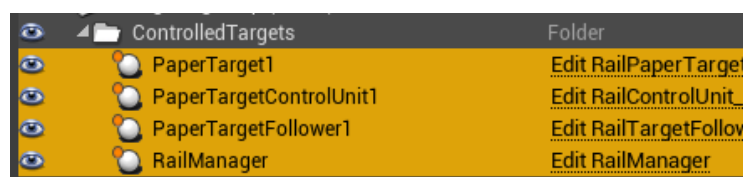
Since one of the key tasks of the coursework is refactoring our Character Blueprints, let's practice refactoring with our Control Unit to create a new object with different behaviour.

1. Again, find **RailControlUnit_BP** in the Content Browser – right-click and select Duplicate, renaming the Duplicate to **SpawnControlUnit_BP**
2. Drag it into our Level and position and rotate it so it is next to our Original Control Unit.



NOTE: I have applied a custom material to illustrate the different unit (*yours will be the same colour as the original*).

3. The four items to the original control unit were placed in a folder called **ControlledTargets** – you can remove all four from our Level now, by selecting them in the World Outliner and pressing Delete. If you get a warning about references, you can ignore that, as you are deleting the asset which has the reference anyway.



4. Next, we need to modify our Character's so that they can interact with our **SpawnControlUnit_BP**.
5. Open **ProjectileCharacter** and find the **InputAction Interact**. Find the code where the Hit has been detected, you will see a cast to **RailControlUnit_BP**. To the Cast Failed execution pin (the white arrow) wire up a Cast to **SpawnControlUnit_BP** instead – make sure you call **OnTraceDetected** on this object and not the RailControlUnit.
6. Repeat this process on the RayCastCharacter as well – this will hopefully show you the similarities with the Characters in the process.

This is an example where again **Polymorphism and Inheritance** could have helped us make the interaction code cleaner – if we created a common class for ControlUnit and then Inherited both Spawn and Rail Control Unit from that, we could then just cast to the common class and the code would handle both types of units.

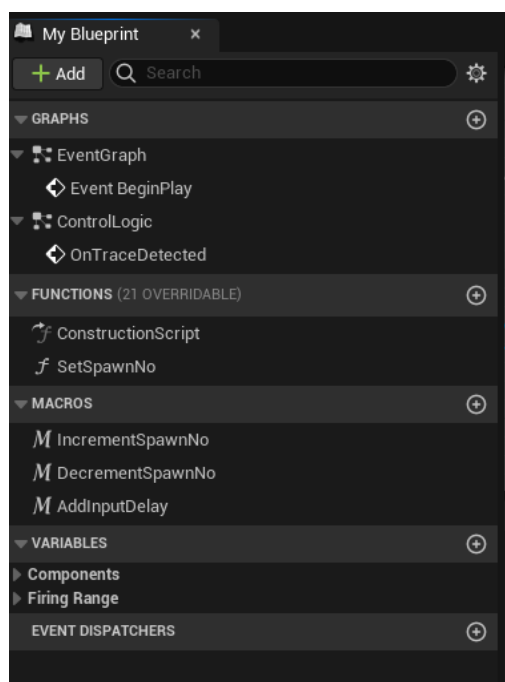
In this case, it is not an issue, since we are removing the RailControlUnit from our game, but hopefully, you can see that this is a better software engineering approach.

You should always be looking for these relationships in UE5 – it allows us to make more re-usable blueprints. Alternatively, we could have created an interface and used that approach as well.

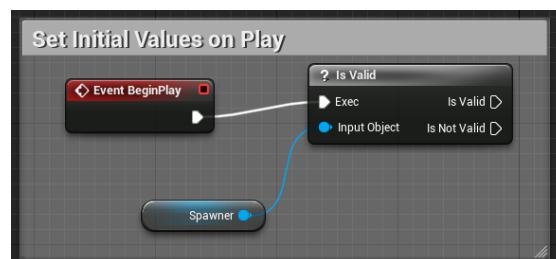
OK, so we can interact with our **SpawnControlUnit_BP** and currently it is performing the same code as our original unit – let's start to **Refactor our code**. As the name suggests, this control unit is going to trigger some objects to dynamically spawn into our Level.

7. Back in our Spawn Control Unit Blueprint, delete the **InterpolateToPos** macro. Rename the Increment and Decrement Macros to **IncrementSpawnNo** and **DecrementSpawnNo** respectively. Rename the Function SetDefinedDistance to **SetSpawnNo**. Delete the Event **GetCurrentLocation**.
8. Delete the variables **DefaultX**, **DefaultLocation** and **CurrentLocation**.
9. Rename MinDistance, MaxDistance and CurrentDistance to **MinSpawnNo**, **MaxSpawnNo** and **TargetSpawnNo** respectively. Rename ControlledActor to **Spawner**.

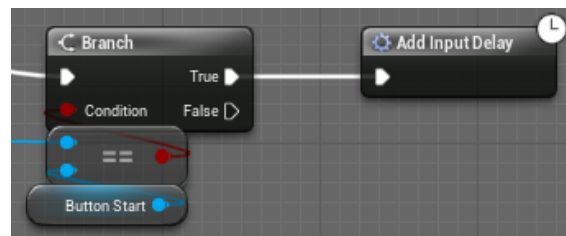
Your Blueprint should look more like this now:



10. Next, we need to update our two main events. Alter **BeginPlay** so it looks like the following:

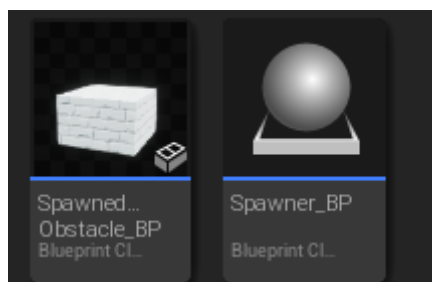


11. Alter **OnTraceDetected**, so that only the following is performed when the Button is equal to our Button Start:



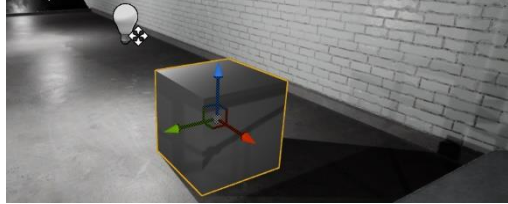
12. Compile, Save and close the Blueprint.

13. We now have the Skeleton of our SpawnControlUnit, so let's add in our Spawn System – for this we need two Blueprints – one for the object we are spawning and another to spawn the objects. Inside our **GameBlueprints** folder, create a blueprint that extends from **Actor** called **Spawner_BP** and a second Blueprint which extends from **StaticMeshActor** called **SpawnedObstacle_BP**.

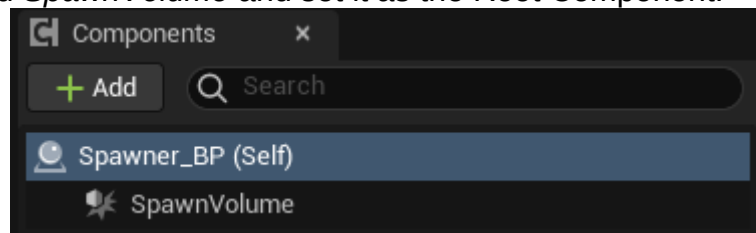


Step 3: Creating our Spawn System

1. Open **SpawnedObstacle_BP** and find the StaticMeshComponent in the component view. In the Details view, set the Static Mesh to the **1M_Cube** mesh. Under Physics, Set **SimulatePhysics** to True. Save and Compile. Test you can create an instance of this, by dragging it into our Level. When you run the game, it should fall to the ground.



2. Open **Spawner_BP**. We'll start with our Components – add a new **BoxCollision** Component called *SpawnVolume* and set it as the Root Component.



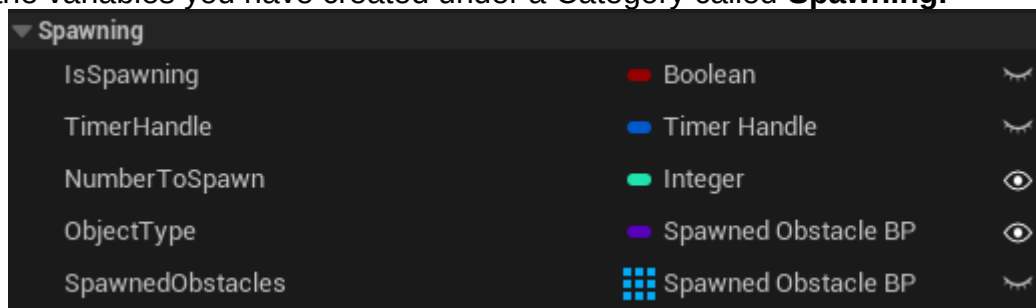
3. Next, we'll add in our Class Member Variables. Create a Boolean variable called **IsSpawning**. Create a variable called **TimerHandle** whose type is a Timer Handle (remember to use the search to help you).
4. Create an integer Variable called **NumberToSpawn**. Compile and set its default value to 50.
5. Next, we need to create a variable called **SpawnedObstacles** – this is an Object Reference to our **SpawnedObstacle_BP** Blueprint. Use the Button at the end of the variable type to change this to an array.



6. Next, create a variable called **ObjectType** – the type of this is also a **SpawnedObstacle_BP**, but this time, it is a **Class Reference** to our Blueprint. Make this **Instance Editable**.



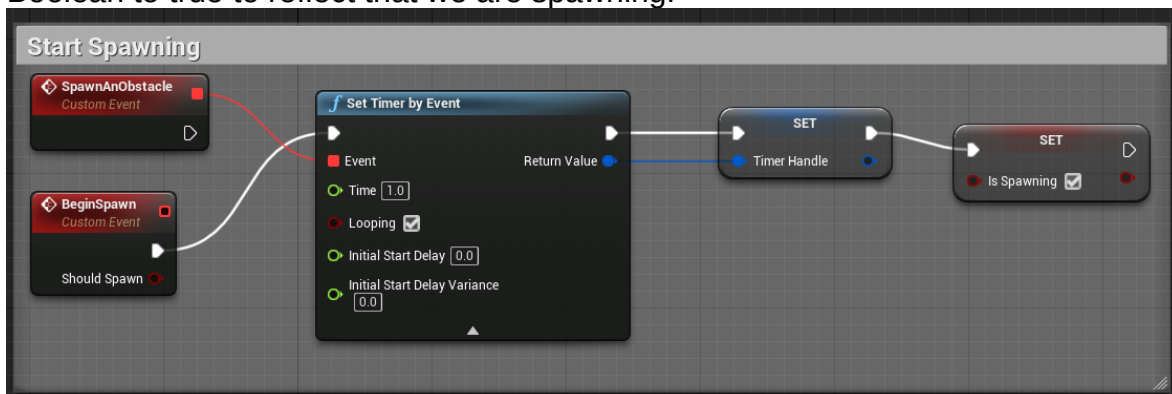
7. Put all the variables you have created under a Category called **Spawning**.



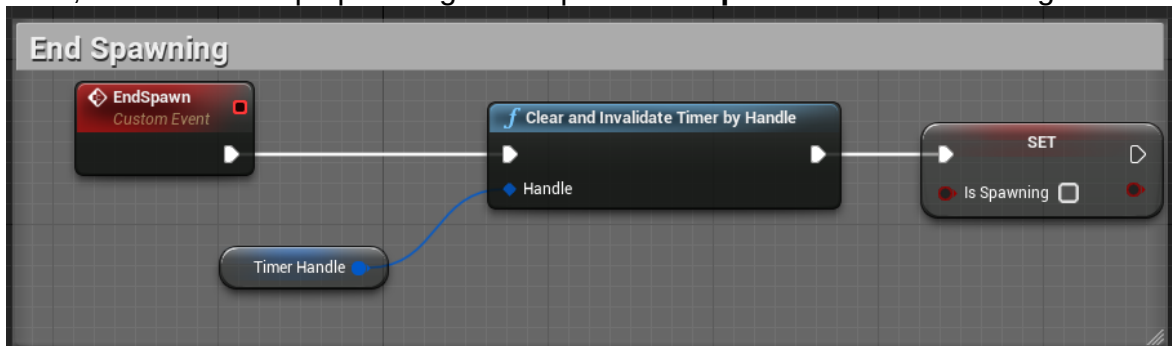
8. That's the data we need, now we need to implement our functionality – we will need three events for this. Start by creating a new EventGraph called **SpawnLogic**. Separate Event Graphs help us group events together and can make editing easier.



9. Open this EventGraph and inside the Graph, right click and type the word Custom – you should see a node called Add Custom Event... use this to create three Events called **SpawnAnObstacle**, **BeginSpawn** and **EndSpawn** respectively.
10. To complete **BeginSpawn**, add the following nodes – notice that it uses the Red Pin on the Event itself – this is another example of Delegation, just like our Event Dispatchers – in this case, it will just set the Timer to call SpawnAnObstacle every second. We associate timers with a handle so we can stop, clear, pause and restart them. Finally, we set our Boolean to true to reflect that we are spawning.



11. Next, we want to stop spawning – Complete **EndSpawn** with the following:



This time, we are stopping the spawning by clearing the timer and setting our Boolean to false.

12. Next, we want to complete our **SpawnAnObstacle** event – this is what is going to run on our timer.

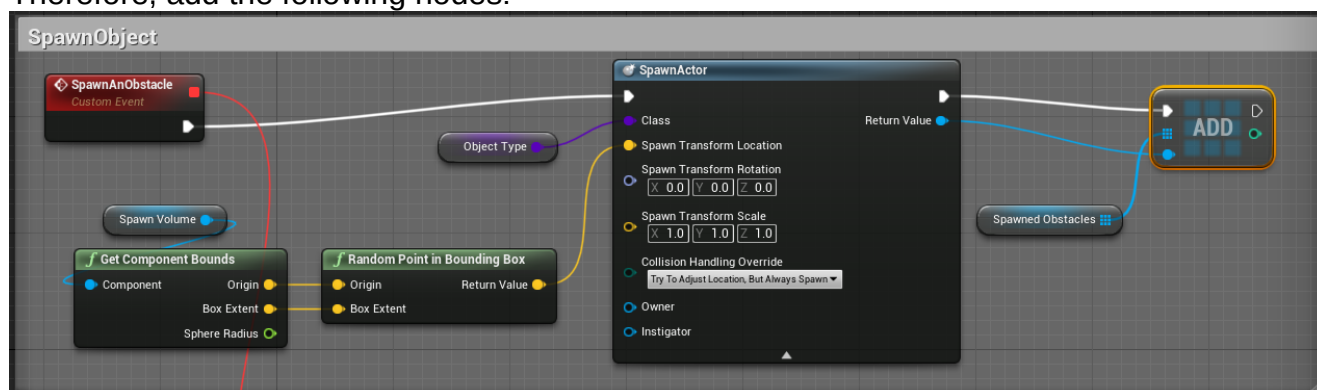
We just need to tell it what to spawn and where.

This is where our class reference comes in – a Class reference stores the name of the class and its relationship – **SpawnActor From Class** uses this to determine what type of object to spawn.

The process of where to spawn is going to use some randomness – we will use our **SpawnVolume** to determine the location – that means that we can change the size and position of the box in the editor, and this will change where items are spawned.

Once the object has been spawned, we keep track of it by storing it in our array of objects – remember, Object References are basically pointers to instances of a particular class (*and therefore an Object*).

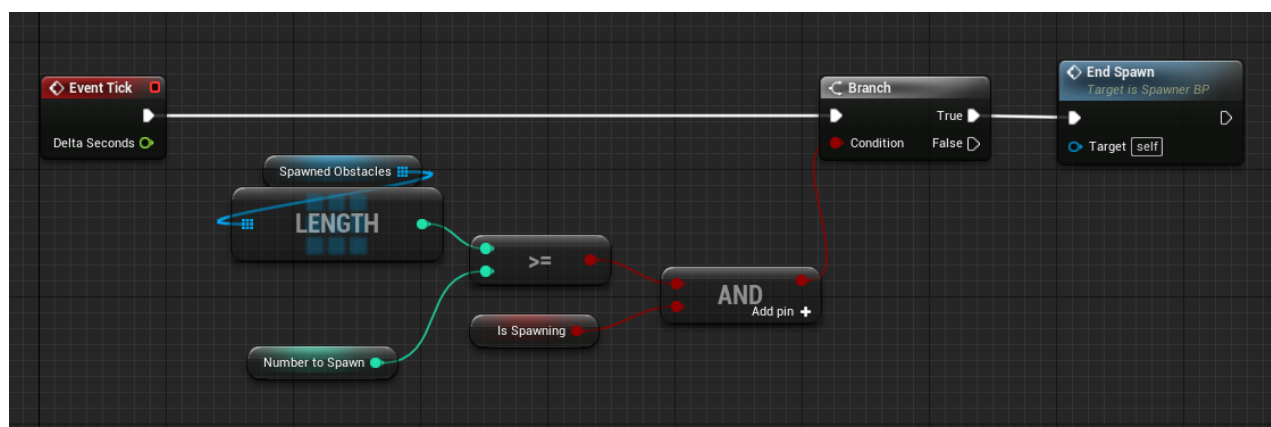
Therefore, add the following nodes:



13. Finally, we need to add in the behaviour that will tell our Spawner when to stop spawning – this is where our NumberToSpawn comes in – we are simply going to see if the size of the array is greater than or equal to the number we want to generate – once we reach that point we stop spawning by calling EndSpawn.

To stop this running every time we reach the maximum number of objects, we need a logical AND so that IsSpawning also has to be true. Therefore, once it has been true once, the next time it won't fire since we know End Spawn sets IsSpawning to false.

We need to check this periodically, so we'll do this the easy way. Override the function **Tick** (or open Event Tick if it is already there) and add the following code:

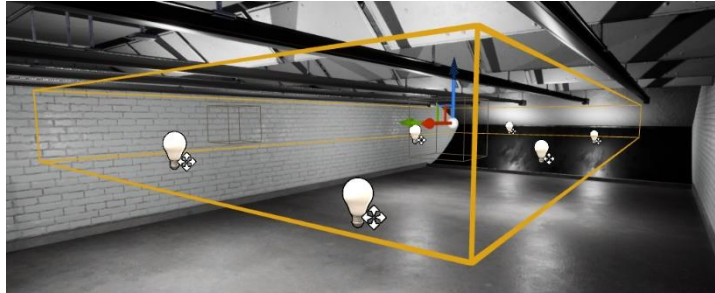


14. We are code complete now, so just **Compile and Save the Blueprint**.

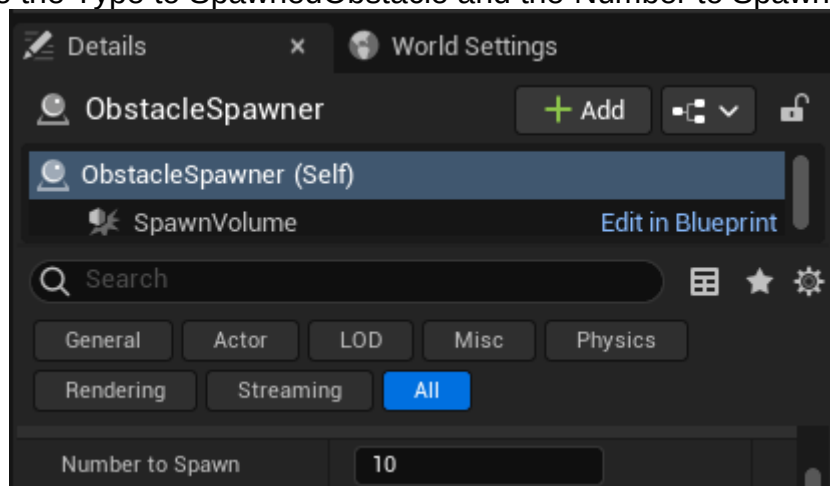
Now we have our Spawner object, we just need to add it to the level and control it from our Control Unit.

Step 4: Integrating our Spawner with our Spawn Control Unit

1. Simply Drag an Instance of our **Spawner_BP** into the Level. Name the instance **ObstacleSpawner**.
2. In the Details view, select the SpawnVolume component and under Shape, change the Box Extents using the sliders to scale the volume – position the actor so it is just below the ceiling of our Range area.

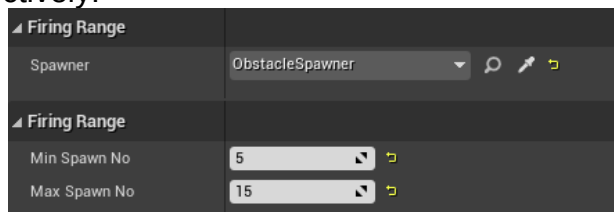


3. Select ObstacleSpawner (Self) and then in search details type **Spawning**. This will allow us to set our Instance Editable data – in this case, it is the number we can spawn and the Type. Change the Type to SpawnedObstacle and the Number to Spawn to 10.

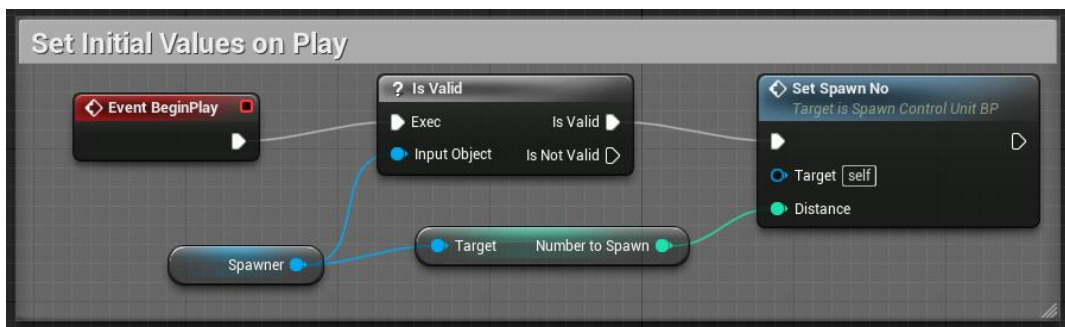


4. Select Edit Blueprint Drop down (next to +Add) and choose “Apply Instance Changes to Blueprints”. This takes the values that we have set in the editor and uses them as the default values (changing our Constructor values) – that means next time we drag a SpawnVolume in, it will default to our Obstacle class, Number to Spawn will be 10 and the Volume box will fit our Room better (you can test this by Saving and then dragging a second instance in – delete it once you have done this though).
5. Our final steps are to wire up the Spawner to our control unit.

Select the Control Unit in the World Outliner that you added earlier. In the details view, find the Firing Range category and set the Spawner to our ObstacleSpawner and the Min and Max to 5 and 15 respectively.

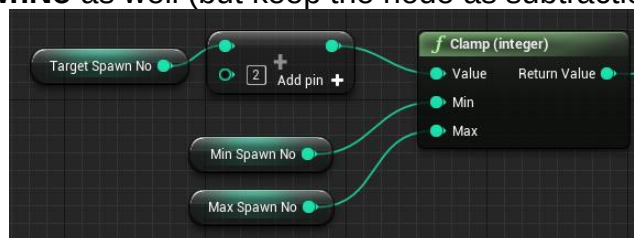


6. Open our **SpawnControlUnit_BP** in the Blueprints Editor. Locate the variable Spawner and change the reference from an Actor to a **Spawner_BP** Object Reference. Now we can access our functions and data of our Spawner_BP object.
7. Let's start with **BeginPlay**. Extend the blueprint to the following:



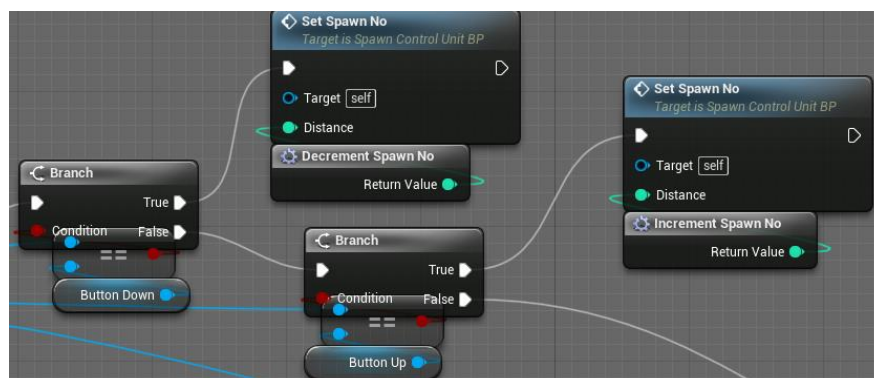
We are reading our Number to Spawn and setting it as our Target Number via our **SetSpawnNo** function.

8. Open **IncrementSpawnNo** and change the Clamp node to the following. Do the same with **DecrementSpawnNo** as well (but keep the node as subtraction):

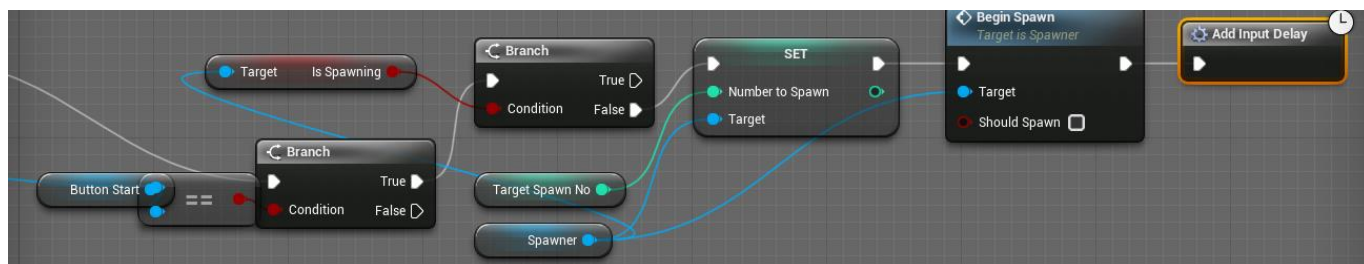


Now we are going to jump our value in twos (e.g. 10,12,14 etc).

9. Finally, we just need to complete our Button handling code in **OnTraceDetected**. Make sure that the Down and Up events are setting the value of **TargetSpawnNo** in the correct branches via our **SetSpawnNo** function (you can change the parameter name if you wish):

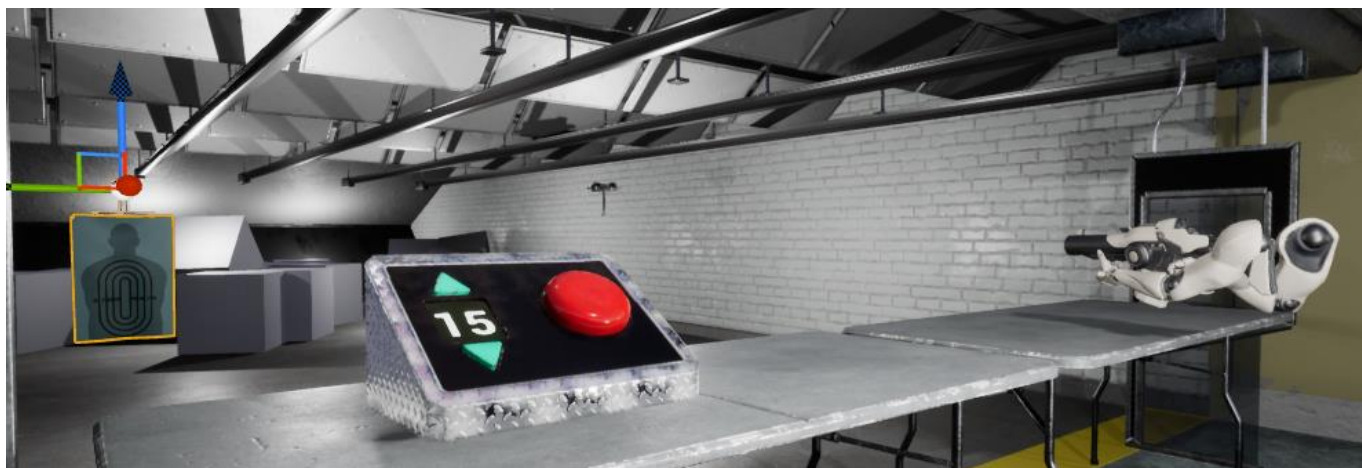


Change our Button Start code to the following:



We only want to spawn if we are not spawning already, so if `IsSpawning` is false, we update our `Number to Spawn` on our **Spawner** object to be the same as the current value from our Unit and then **BeginSpawn**.

10. Compile, Save and Run your game – the unit should display 10 by default and allow you to change the values up and down between the range 5 – 15, going up and down in increments of 2. When you press the button, this should start spawning our cubes.



Hopefully, your interactions with the control unit are working – you can trigger the spawning and using either the Projectile or RayCast Character and when you collide with the boxes they can be pushed by the ray cast bullets and destroyed by the projectiles.

If it is not functioning, make sure you have correctly implemented the logic for the character interactions in the Character class and that the input parameters are wired up correctly.

Task 3: Enhancing our Target Systems via Timelines

In this final task of the Tutorial, we are going to work with **DynamicObject** system to show a better way of Animating our Targets; for this, we will be using **Timelines** and different types of curves. We will also add a couple of instance variables that allow us to provide some variation.

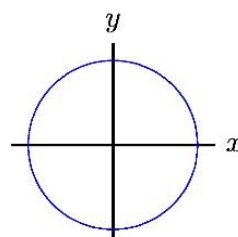
Curves are basically 2D Graph Plots, which we can use for various purposes – in our control unit, we are using the curve to arrive at a value between 0 and 1 to use with interpolation. We can also use them directly with Positional and Rotation Data.

Before we work on the code in UE5, let's just look at a bit of background mathematical theory to put them in context. These plots are a graphical way of depicting **Parametric equations**. A **Parametric Equation (or system of equations)** is where we separate out dependent relationships by introducing functions of an independent variable.

For example, the equation of a unit circle is $x^2 + y^2 = 1$ (we know this through Pythagoras, since, in every possible triangle formed for different x and y values, the radius is 1, and this is the hypotenuse). Therefore, x and y are dependent on each other (e.g. $y^2 = 1 - x^2$ and therefore, $y = \pm\sqrt{1 - x^2}$ – there are two solutions for y , since x^2 has positive and negative answers (e.g. $4 = 2 \times 2$ and -2×-2 so both -2 and 2 are answers for $\sqrt{4}$).

If we solved this equation for every point where $-1 \leq x \leq 1$, then we would end up with every point on the circle.

This can be simplified by instead making equations for x and y which are independent of each other – we introduce a new variable t – in this case the angle between the range: $-\pi \leq t \leq \pi$ and a functions for x and y which use t , so we can work out the x position of the radius and the y position of the radius, at every angle t . We know from Trigonometry, these are:



$$x(t) = \cos(t)$$

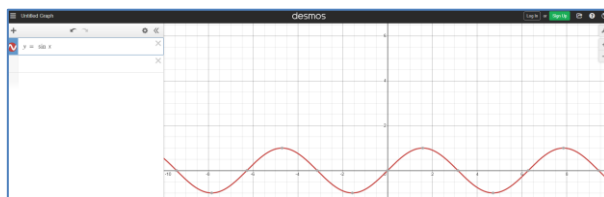
$$y(t) = \sin(t)$$

And thus, the x and y values can be calculated separately. Parametric Equations are commonly used in animation because it is very easy to make t a function of time. In the above case if t was time, then what we are calculating is the point on circle's perimeter we are at for a particular moment in time – as we increase time, we would work our way around the circle.

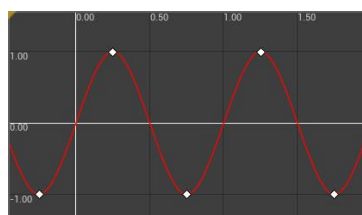
This is very easy in the case of our angles; we just need to map our time to our angle value. We do this by specifying the rate of animation – e.g. if we passed through 360 degrees / 2π radians in a second (one revolution of a circle), then at $t=0$, our angle is $-\pi$, and at $t=1$, our angle is π and therefore we have covered 2π radians during this time.

If you are unsure, experiment with this interactive demo to see the relationship between the two <https://www.mathsisfun.com/algebra/trig-interactive-unit-circle.html>

I also recommend the Desmos calculator, for picturing how particular functions may look.
<https://www.desmos.com/calculator>

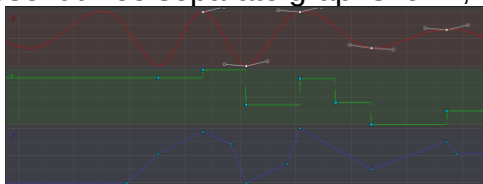


UE5 Timelines, therefore, as the name suggests, are a way of visually constructing parametric equations, over time. We do this by creating key frames on an **x-y** graph plot – y is our value we are calculating, and x is the time. Below is **y(t) = sin(t)** as a UE5 timeline. There are many types of wave form we can create using the curve editor.

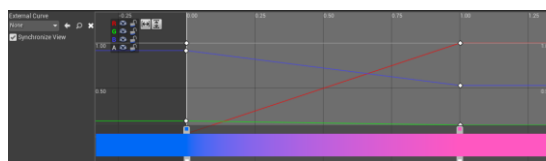


There are four main types of curve we can create:

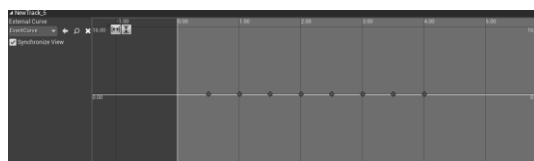
- **Float curves**, which represent a single value on the Y axis and time on the X axis. The above image is an example of a float curve.
- **Vector curves**, which represent three separate graphs for X, Y, Z overlaid onto the Y axis.



- **Colour Curves**, which represent each of the colour values (R,G,B,A) as separate curves, which join together to form specific colours (UE5, lets us just choose a colour from a picker to get values for RGBA):



- **Event Curve:** Not actually a separate curve, instead it uses a float curve to send out events – it ignores the actual values of the curve – it just fires every time it encounters a key frame.



OK, so that is the theory behind timelines and curves, let's put this to use with our Targets – we will be creating various blueprints, which extend from DynamicTarget, to illustrate different ways of using timelines.

My advice is after completing this tutorial, **to experiment as much as you can with them** – the Curve Editor can be a bit cumbersome to begin with, but once you get used to it, it is enormously powerful – I use it for so many things beyond animation (e.g. a float curve for weapon damage based on distance – give it a distance value and look up the damage; sharpness decay on a sword – the number of hits is the sword; dynamic scoring based on completion time; XP Level up – use Event Curves to increase level when a key frame is hit; open and closing doors; unlocking new areas on a map etc – there are almost infinite ways of using them for smart gameplay!).

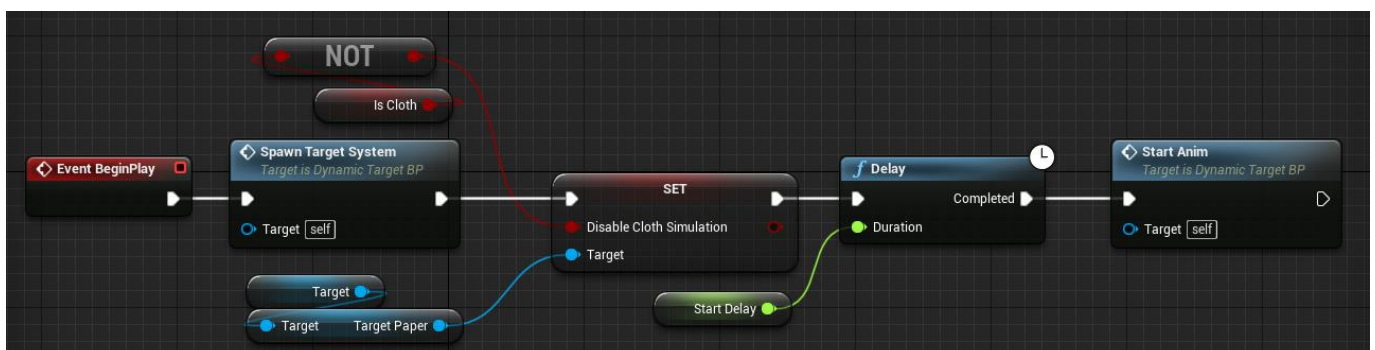
Step 1: Extending Dynamic Target to incorporate Timelines and customisation.

Let's start with adding some variety to our Dynamic Target system – we want each instance to animate at a different time and to alter the behaviour of the Target – the factor that will impact the types of movement we can support the most is the Cloth surface of the Target.

As the surface is rotated or translated, this will recoil under physics, but is also affected by the Physics Engine's gravity system – to make the target rigid, we need to disable cloth.

Different types of target movement will benefit from these two options.

1. Locate the **DynamicTarget** blueprint and add in two variables:
 - *StartDelay*, which is an instance editable **float** value.
 - *IsCloth*, which is an instance editable **Boolean** value.
2. Compile the Blueprint and set the defaults to **0.0** and **true** respectively.
3. Next, to allow for the delay, we need to add a new event which is going to trigger the timeline to commence. Open the EventGraph and add a new custom event called **StartAnim**.
4. We need to incorporate these variables and events into the setup of the Actor – Locate the **BeginPlay** event and alter it to the following:



We are using the Boolean to set the value of the function to disable cloth simulation – we use a NOT node to flip the Boolean, since the function is disabling the cloth, therefore IsCloth is true becomes false, and therefore Cloth simulation is not disabled.

This is a property of the TargetPaper component of our Target, which we access via our Target variable, set up in Tutorial 08.

We then use the Latent Function **Delay** to call our new Event after a certain amount of time has elapsed – this time is from our variable we added in Step 1.

The defaults we have chosen are such that the Target will behave exactly as before by default – the function has a delay of 0.0 and therefore will be called immediately, and the cloth value is true, so we will have cloth simulation enabled.

5. Finally, to make our Positioning more accurate, we are going to modify the Box component to be smaller – select this component and under Box Extent, change the X, Y, Z values all to 2.0. Save and compile the Blueprint.

We are ready to create some Timeline based objects – so why haven't we put the Timeline in this class?

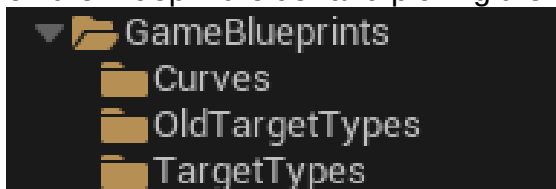
Blueprint Timeline nodes are a component, but when we inherit the component, we lose access to some of the key properties – therefore, to make it easier to use, we will add the Timelines directly to the children instead.

This technically means we are duplicating data that should be common, which is bad software engineering, but in practice, it is easier in this situation due to the way the timeline editor works.

Step 2: Rotating with Timeline Curves

Let's start by creating our first child class that is going to be timeline based – this will use Rotation.

1. Open the GameBlueprints folder in the content browser and add three new folders inside this – Curves, TargetTypes and OldTargetTypes. The easiest way to do this is right clicking on the Blueprint folder and picking the New Folder option:

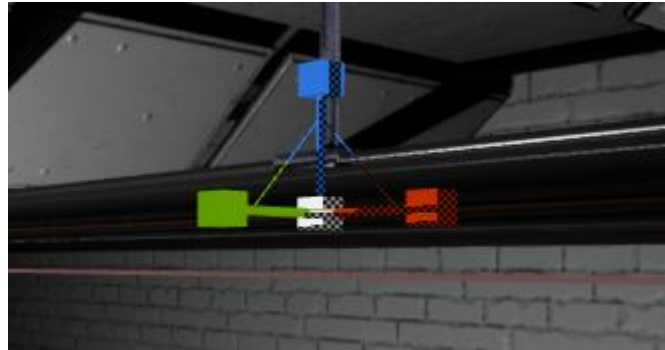


2. Drag the **MovingTarget** and **RotatingTarget** Blueprints inside the OldTargetTypes folder.
3. Drag **TimeCurve** into the Curves folder. Right-click and rename it **CU_TargetCurve** (Control Unit Target Curve) as this curve is used by our Start Button of the Control Unit, as discussed in Task 2.
4. Right-Click on **DynamicTarget** and select 'Create Child Blueprint Class' – this is the easiest way of directly inheriting from a custom blueprint. Move this Blueprint class into the TargetTypes folder and rename this class **DT_SnapRot_Y**. This stands for *Dynamic Target – Snap Rotation – Y Rotator Axis* – we are going to create a class that will create a target that starts folded up and snaps into a vertical position and stays there before folding back up again.
5. When we move assets in the Content Browser, often it doesn't update other Blueprints code that refer to them – instead it adds some text to an editor config file that stores a

mapping between the old location and the new location – this is known as a **Redirector**. This is fine during development, but really, we want to get rid of these.

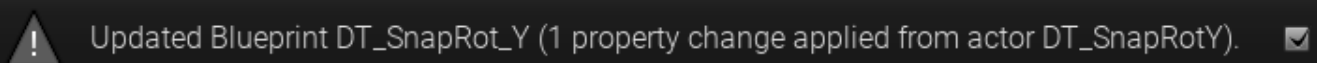
***To do this:** Right Click on the **GameBlueprints** folder and select 'Fix Up Redirectors'. Repeat this for each of the three sub-folders we created as well. After we have fixed up the redirectors, select **File -> Save All** to make sure all our Blueprint changes are saved.*

6. Drag our new blueprint into the level and position so it is dangling from one of the rails.



7. We also want this target to be rigid. Change the **IsCloth** property in the details view to False, and then select **Edit Blueprint -> Apply Instance changes to Blueprint**, to make this become the default value of this child blueprint.

You should get a message that this change has been made, in the bottom-right of the editor:



Run the game – you should have a static target that is now rigid in your level.

8. Open **DT_SnapRot_Y** in the Blueprints editor. Drag an execution pin from Event StartAnim (You may need to override the function from the function - override menu) and type 'Time' – use the 'Add Timeline...' option. Use F2 to rename this timeline **FollowerTL**. Wire up the **StartAnim** Event to **Play from Start** – this will begin the timeline.
9. Double Click the Timeline Node to open our Timeline Editor. Click the Add float track button on the menu bar – this is the first option:

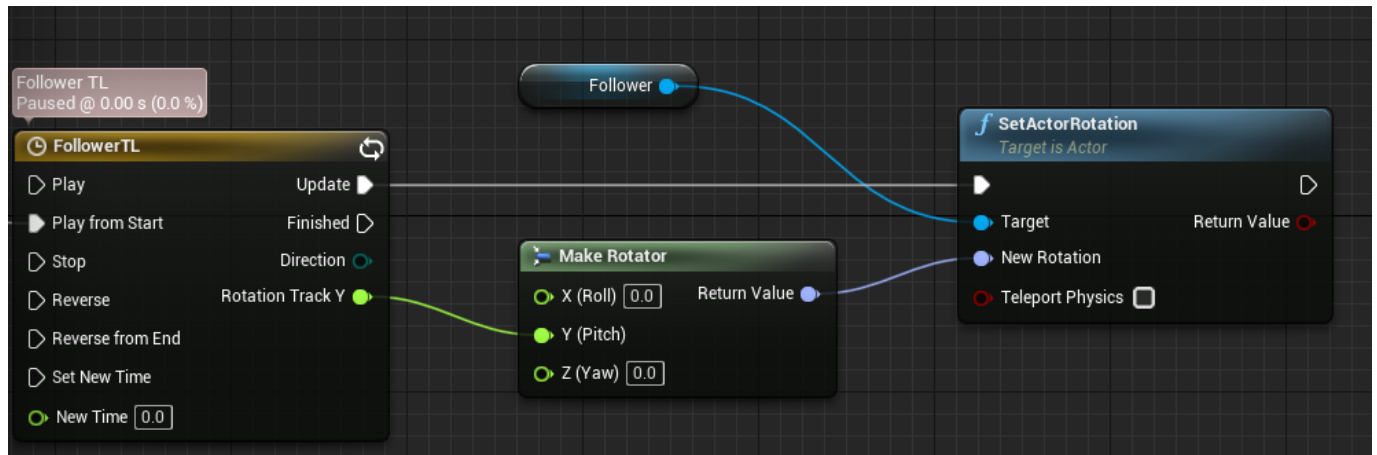


Name this Track **RotationTrackY**. This will be our Rotation curve.

Right Click inside the Graph portion of this new track and select 'Create External Curve'. Select the Blueprints/Curves folder and name this Curve **RotFlipCurve**. You can close the timeline editor for now.

10. You should see that the Track has been added to the Right-hand side of the Timeline Node – this is the output of the timeline at a given time (e.g. what the Y value is at a point along the X axis). We will complete the logic for the timeline rotation first.

Using the following nodes, wire up the timeline's Update function as follows:



We make a UE5 Rotator, which represents Yaw, Pitch and Roll of a 3D Object (corresponding to the rotations of a Fighter Jet) – we want to control the Pitch. The output of this is then used as the input to our Rotation value of our Target Follower Node – if you are finding the getter hard to locate, type in **FiringRange** when you right click to create a node – using the categories always makes it easier to locate what we want.

11. Open the Timeline Editor. Set the Time of this Curve to 9 seconds long and for it to Loop:



12. Next to the curve, click the Spyglass icon, which will locate it in the content browser and open RotFlipCurve in the Curve Editor.

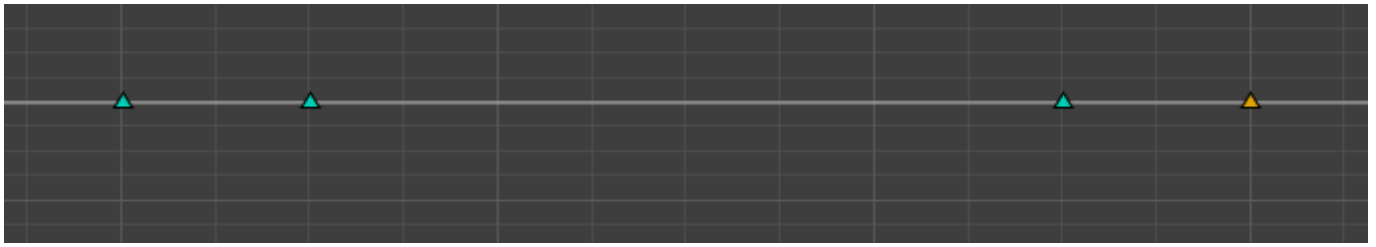
The Curve Editor can take a little bit of practice to get use to – it's not the most intuitive. The process involves creating key frames and setting the type of interpolation we want to use between these frames, to build up our curve.

Set a key frame at 0,0 by finding this point and pressing Enter, or the middle mouse button to insert a frame (use the mouse wheel to zoom in/out). If your values are slightly off, you can adjust them in the Menu bar:



Change both fields to 0. The left-hand side is time, the right-hand side is the Y value.

13. Next, add a key frame at 1 second, another at 5 seconds, and another at 6 seconds, all with a Y position at zero.



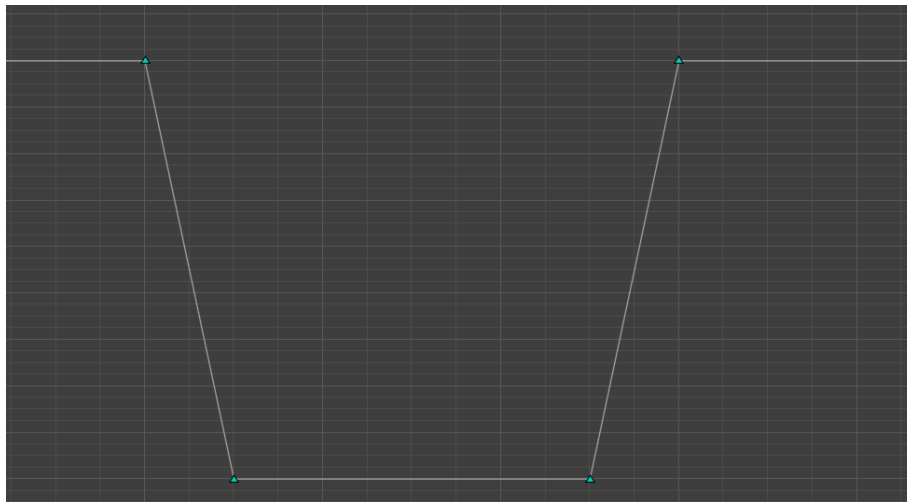
This is the easiest way to get started with building a curve.

- 14.** To make it easier to move only the position on the Y curve, we can change the UI to lock one axis.



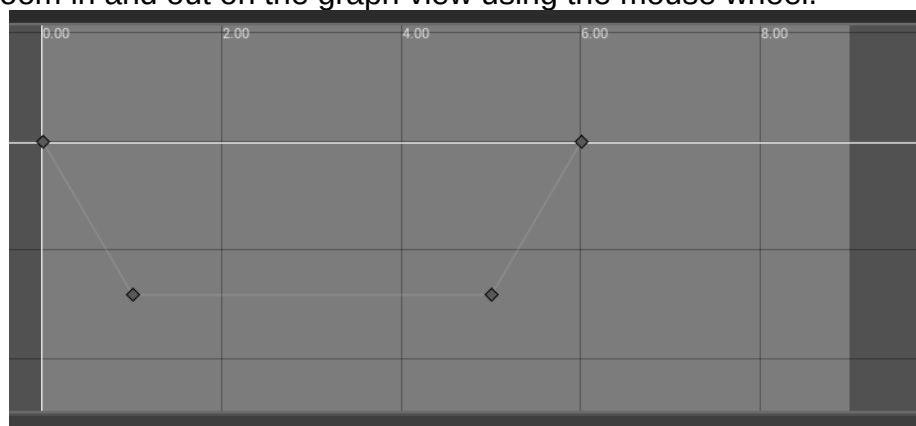
Press this icon to change the lock to Y only.

- 15.** Move the Key frames at 1 seconds and 5 seconds so that they have a Y Position of -90. You should end up with the following:



We've created a simple curve that will linearly interpolate (straight-line interpolation) between our key frames. The values for times less than 0 and after our 6 seconds will continue indefinitely at 0.

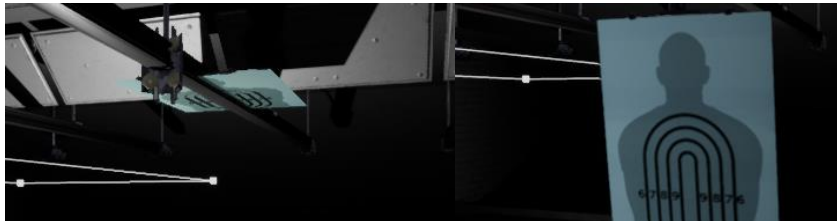
- 16.** Save the curve and close the Curve Editor and re-open the Timeline in our Blueprint. You can again zoom in and out on the graph view using the mouse wheel.



The light grey view is the portion of the curve that will be used by the timeline 0 – 9 seconds.

Therefore, we will start unfolded, at 1 second fold up (-90 degrees), spend 4 seconds folded up, spend a second unfolding (back to 0) and then spend 3 seconds unfolded before starting the process again – because our start and end points are both 0, this curve can be looped easily – make sure you are careful of this when using looping, otherwise you might see an unnatural jump in rotation!

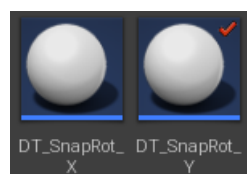
17. Close all editors and compile and save the changes. Run the application to test that our animation is working:



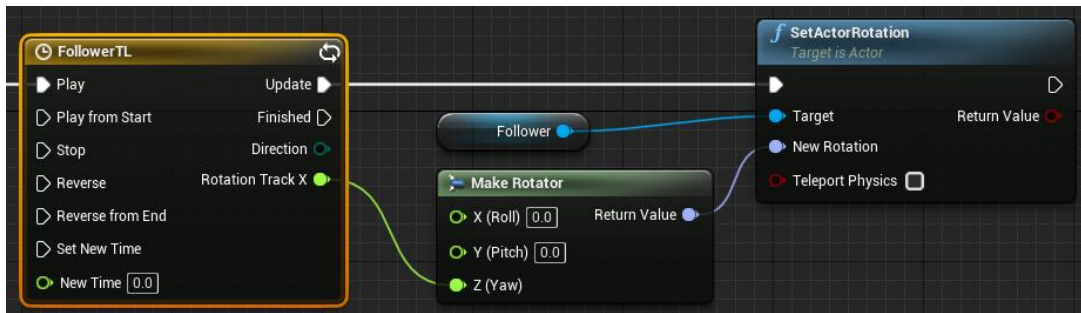
18. Experiment by dragging a couple more instances into the level and setting different positions and delays on each instance.



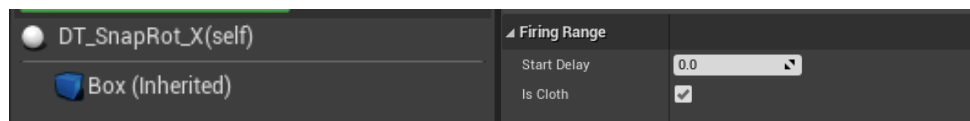
19. Next, we will create a different variant of this rotation. This time, rotating along the Yaw axis of our Rotation. Duplicate the DT_SnapRot_Y blueprint and name it DT_SnapRot_X.



20. Open this Blueprint in the Blueprint Editor. Open the Timeline Curve and on the Rotation Track, right click and then choose the option to rename – change it to **RotationTrackX**. This will still use the same External Curve as the other node – this is our way of easily sharing curves between different animations – the timeline belongs to the blueprint so therefore is different – we can change settings such as looping or effects and it won't affect this timeline.
21. Change the **Make Rotator** Node so that this Track is going into our Yaw position instead of Pitch.



22. Select the **self** node at the top of the components view. Then, using the details view, set the **IsCloth** variable to True. This is the alternative to setting the value on an instance and applying instance changes to blueprint – by setting it here, it will apply automatically to all instances of this Target type.



Once done, you can compile and Save the Blueprint.

23. Drag a few instances of our X Rotating Target into the Level (or hold Alt whilst translating one instance to duplicate it), and again experiment with varying the Delay between each instance.

From our simple use of one Curve, we now have two different types of Target, each with separate behaviours. We also still have an inheritance tree, where our Targets are both children of **DynamicTarget**.

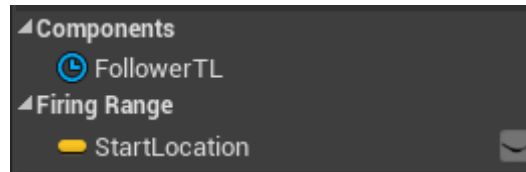


Step 3: Translation with Timeline Curves

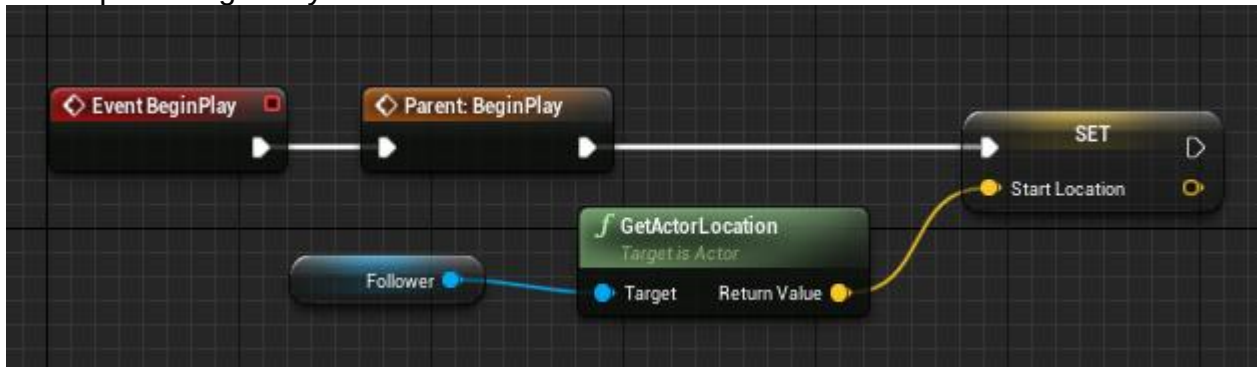
Our next implementation step is to add in a Timeline that moves the object – we will write one like the one from Tutorial 08, so show the ease of Timelines.

1. Duplicate the Blueprint for **DT_SnapRot_Y** and name it **DT_MoveForwards**
2. Alter StartAnim so that everything which is connected to the Update of the Timeline is removed.

3. Add a new Vector variable to the Blueprint called **StartLocation** – put this in the Category **FiringRange** as usual.



4. Wire up the BeginPlay event as follows:



This is an example of extending the functionality of a Blueprint function – the same as we would via a virtual function in C++ from its Parent class. We need call BeginPlay in our Parent class **because** that is where we spawn and set the target and follower. After we have a follower object, we need to use it to get its location and set that as our StartLocation – this way we can make our movement *relative* to wherever the Follower is spawned.

5. Double-click on our FollowerTL timeline node in the **StartAnim** graph. Remove the Rotation Track by right clicking in the space under the Synchronise View action and deleting it.
6. Untick the Loop option – we want this timeline to only play once.
7. Add a new Float Track and name it **TranslationX**. Right click on the Graph to make an External curve and put it in our **GameBlueprints/Curves** folder with the name it **MoveForwardBackCurve**; open it in the Curve Editor as we did before.
8. Add in key frames at times 0, 2, 8 and 10 sections, all positioned at height 0. Between 0-2 we are going to move forwards, stay there, and then return to the start between 2-8.
9. To do this, drag the key at 2 seconds to a height of 100 and the key that 8 to a height of 100.

You should have a curve with the following basic shape. We will do some modifications to this curve since this is too short a distance to travel. To help, we can Transform our Curve.



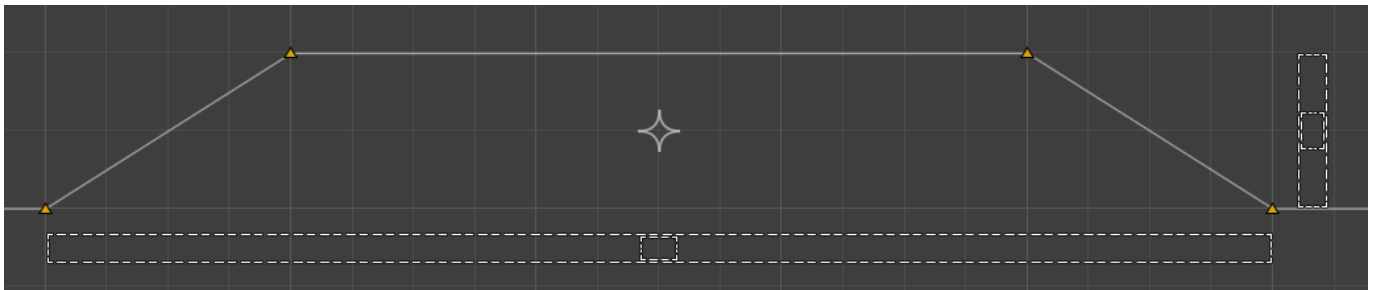
We have a few tools on the Menu bar that can help us modify our curve.



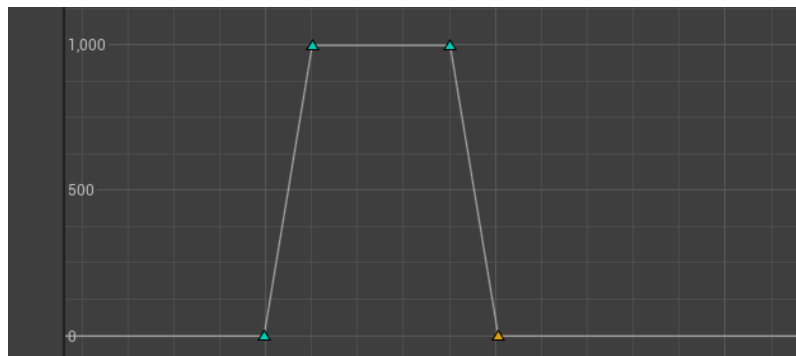
- The first tool is selection, this helps us move individual keys – we also click this to finish using the other tools.
- The second is a transform tool – which allows us to translate and rotate keys.
- The third is a Re-timing tool – this allows us to stretch the keys over time, but also translate it along the time curve – this is good for adding delays into our curves.
- The fourth is a Multi-Select scaling tool – this is the one we want to use.

We could also do this via the second tool, but the fourth is easiest to move sets of keys.

10. Click on the Fourth tool (the wavy lines) or press Ctrl+M to activate it. Click on the Curve Line and you will see the two control sliders appear.



11. In the tool menu in the bottom right, set the pivot centre as the first key. Then Drag the Vertical slider until the curves top line is at 1000 units. You can check this by hovering over the keys. Once you have set this, press the selection tool to return to normal mode:



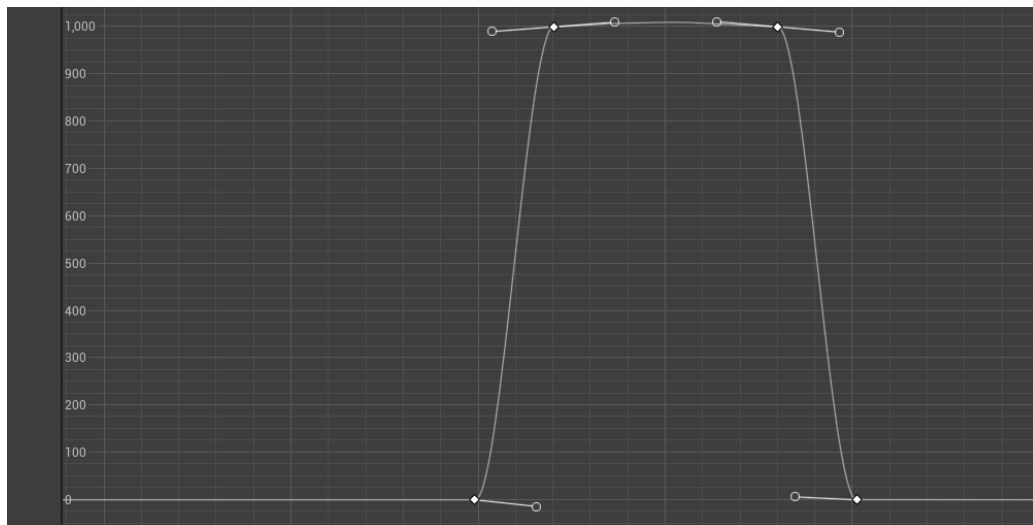
If you need to increase/decrease relative distances or rotation amounts, these tools are the easiest way of doing this.

12. Finally, we want to do something a little different to linear motions. For each key, we can control its interpolation mode – there are five options, all of which are on the menu bar – however, the shortcuts for these are easier, as they are the number keys 1 to 5.



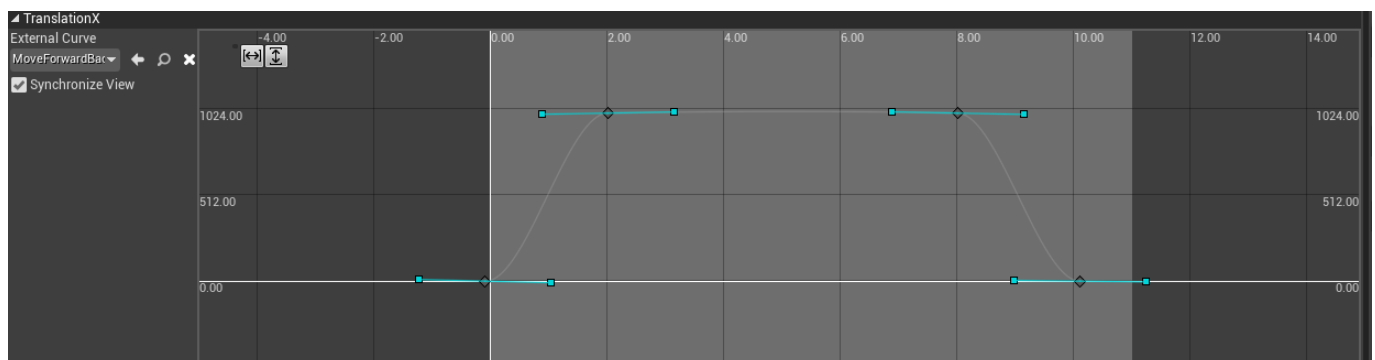
The default option is Linear, which is 4.

Create a slight curve around each point using mode 2 on each of the keys and then adjusting their tangents. What are controlling here is basically the speed and acceleration of our translation. Where a speed steepens, we are increasing speed (accelerating), where we are making it shallower, we are slowing down (decelerating). Linear motion means constant speed to a destination point.

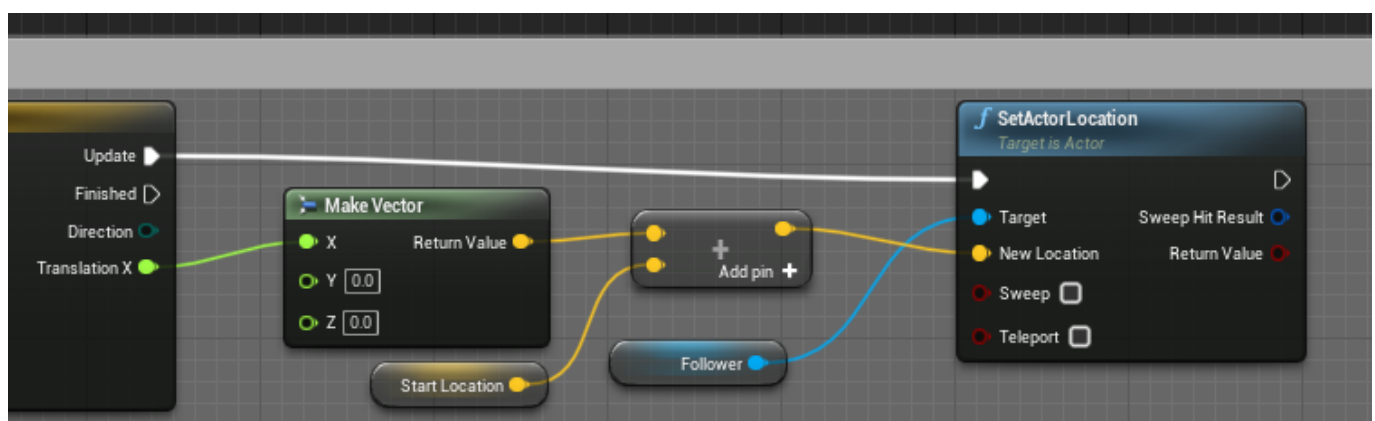


13. Save the Curve and close the editor and return to the Timeline in the Blueprint.

14. Set the duration of the Timeline to 11 seconds.



15. Now all we need to do is wire up this track to our update function to trigger the translation. Add the following nodes to the update.



We are setting the position of our Target to the position from our Track but offsetting it by our StartLocation – if our target starts with an X position of 200, then at time 0 it would be $200 + 0 = 200$ and at time 5 it will be $200 + 1000 = 1200$. We just use this with the SetActorLocation function of our follower to handle the change in position.

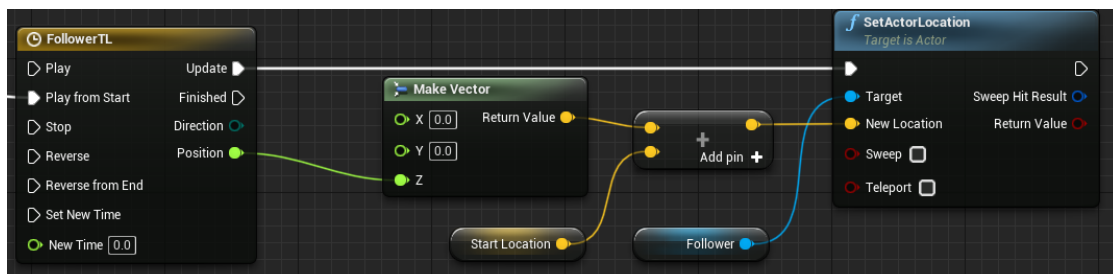
16. Again, drag a few instances and experiment with different delay and IsCloth values to see the different behaviours we can get with this code.



Step 4: Appearing with Float Curves

Our next example is going to use Constant Interpolation. This means an instantaneous change in Rotation or Position (e.g. Teleporting). We can use this in lots of different ways, but in this sample, we will show a creative way of using it.

1. This time, duplicate the DT_MoveForwards Blueprint as the basis, calling the new Blueprint **DT_InstantAppear**
2. Open the Blueprint and then the Timeline. Remove all the existing tracks and add a new Float Curve Track called **Position**. Create an external curve in the **GameBlueprints/Curves** folder called **MagicAppearCurve**.
3. Close the timeline and return to the Blueprint graph. Wire up the Update event of the timeline as follows – you should be able to do this, just by modifying a couple of the nodes already there:

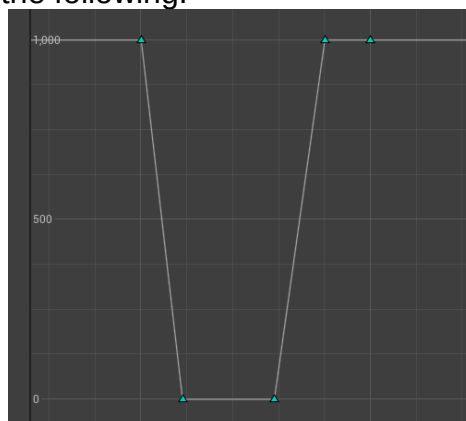


4. Compile and Save the Blueprint and open the newly created Curve in the curve editor.
5. This time, add keys at 0, 1, 3 and 4 and 5 seconds.

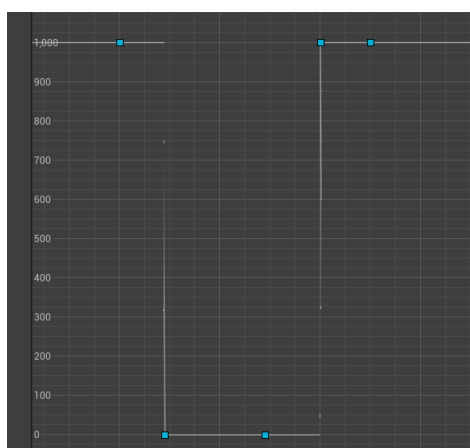
6. Select the 0 and 4 and 5 second keys by holding shift and clicking. The menu is now show multiple values in the time section – this means we can change the Y value for all selected keys. Alter the second box along to 1000.



Your curve should look like the following:



7. Change the Interpolation mode of the first key to constant. Now your curve should look like this:



This means that at the points the line goes vertical, this is an instantaneous change in position, we will go from 1000 to 0, and then back to 1000 immediately. This gives the appearance of teleporting.

8. Right click on the curve line itself and select Post-Extrap. This means extrapolate the curve after the last key frame – we have various options controlling what to do next – most are some form of repeating or continuing along the same Y value or same direction as the last key frame. Choose Cycle from the options and you should now see the curve will just repeat. This is the equivalent of looping the timeline, but what this allows is us to set a finite time on the Timeline itself.

Save the curve and go back to the timeline in the Blueprint. Set the Duration to 15 seconds – this is 3 cycles of our extrapolated curve. The view of the curve in the timeline window might not show the cycles, but it will work in game.

9. Compile and Save the Blueprint and then drag an instance into the level – test this out and you should find the target appears 3 times before permanently disappearing – what we are

doing is putting the target 1000 units up in the sky (Z-Axis) and then back down to 0, relative to the target's position in the level, so it instantaneously drops, then instantaneously raises. This works because our level is closed – there is no way we can exit the room to see this happening – simply moving an object outside the bounds of a level is a great way of changing the interaction of an object if you don't want to destroy it.

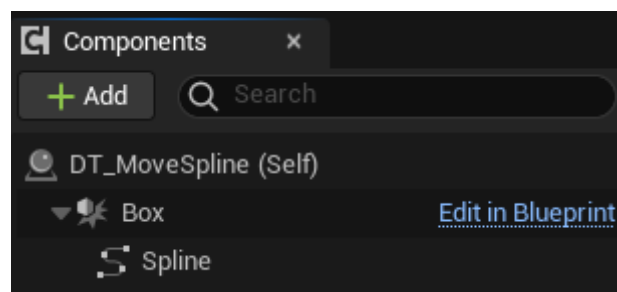


10. Again, test out multiple instances of this with different delay and cloth settings.

Step 5: Using Spline Curves for More sophisticated Motion Paths

Our final example of using Timelines is to not use the curve itself for interpolation of rotation or position, but rather use the curve to drive the interpolation time of another engine method of motion – in this case, it is the Spline Component – Splines are powerful because we can draw exact motion paths in 3D space – they are basically 3D curves with multiple segments and key frames.

1. Splines don't need to know anything about the relative position, because they are attached to the Actor, so when we move the Actor, the spline move with it. Therefore, we can Duplicate our Rotating Target **DT_SnapRot_X**. After duplicating it, rename it **DT_MoveSpline**.
2. Open the Blueprint in the Editor. To the components, select the Box component that has been inherited and select Add Component and search for Spline, which is under the Utility category.

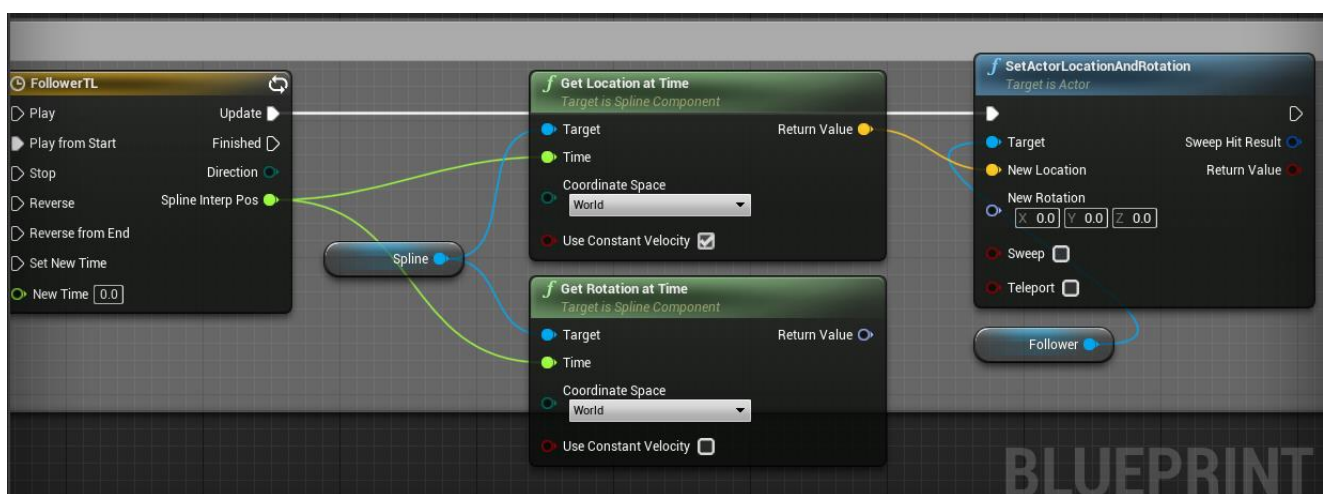


3. Open the Viewport and you will see our Spline Component – by default, one point to the spline has been added for us – we want to remove this, since we will be adding our own points that may end at the location of the Box.



To do this, just select the end point (it's the one that's not in the same location as the Box) by left clicking and pressing the Delete key to remove it.

4. Save the changes to the Blueprint. Open the FollowerTL Timeline and remove the existing tracks, adding a new FloatCurve track called **SplineInterpPos**. Set Timeline to Loop and the Duration to 10 seconds.
5. Update the StartAnim function so that the update of the Timeline is as follows:



We are using the Spline's Get... at Time functions to get the position at a point along the spline, we are also getting the rotation as well.

To set these, we will use the **SetActorLocationAndRotation** Function, to set both at the same time – notice by default we are not going to use the Spline Rotation.

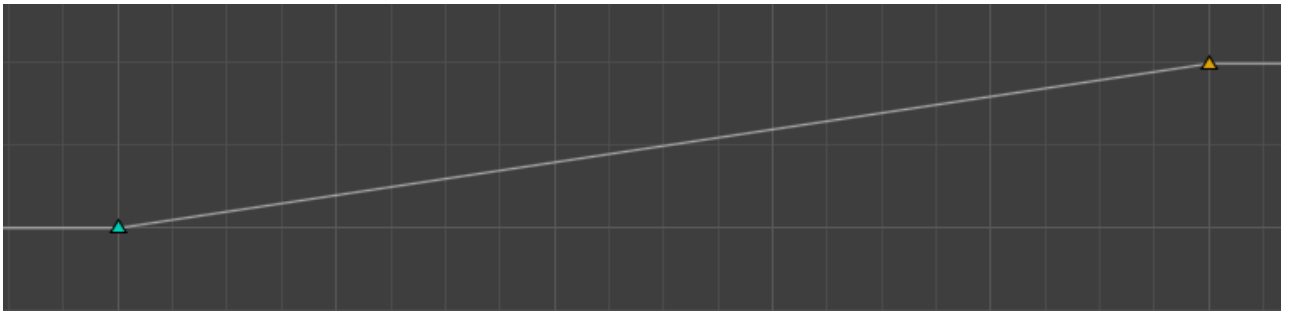
6. The final step we need to implement is to create our Timeline Curve – open the Timeline editor and create an external curve for SplineInterpPos. Name this **SplineInterpCurve**.

Compile and Save the Blueprint.

7. Edit the **SplineInterpCurve** in the Curve Editor.

For this, we want a normalised curve – that is a curve with Y values between 0 and 1, where 0 is the start of the curve and 1 is the end of the curve – this is true for most types of interpolation we will use, whether it is Curves, or the LERP and SLERP functions of a maths library.

Therefore, create a curve that is 0 at 0 seconds and 1 at 10 seconds. Save this curve.

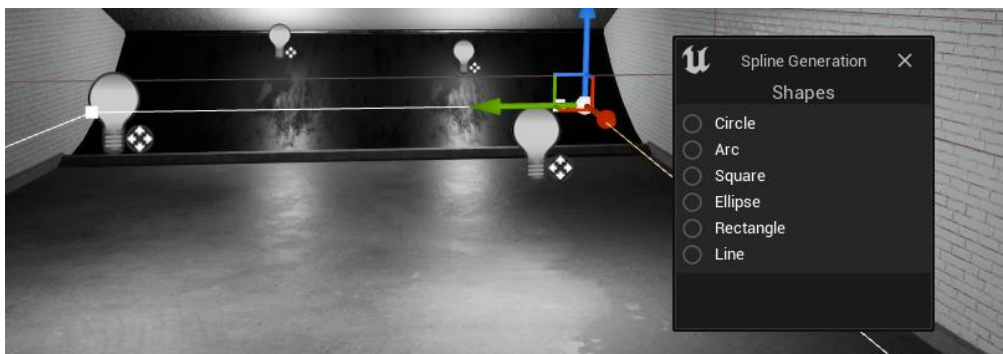


This type of curve is great for interpolation, but also if you want to dynamically change distances covered or rotation rates etc – we simply can multiply it by a value.

8. Drag an instance of the **DT_MoveSpline** Blueprint into the Level and then select it and right click at the centre of the gizmo – this should bring up the Spline Editor windows – again, Spline Editing takes some practice, but UE5 has recently added a feature which makes this much easier.
9. Select the **Spline Generation Panel** option.

We will create a rectangle-based spline, by selecting the Rectangle Radio button. Use the Shape parameters to change the position of the Spline's spline points.

Close the Panel when you are happy with your spline.



10. Run the game and test that the target is now following the Spline. It should carry on following the path.



Step 6: Profit! Applying the concepts for your own gains!



OK, so I have shown you multiple different ways of using Timelines and Curves to dynamically animate the position and rotation of our Target – now it is time for you to experiment and practice with these concepts – from this, you should be able to create some interesting and complexity variety of targets.

- Try out different curves with our existing samples, by either duplicating the Blueprints and changing the curve used, or by adding new tracks to the Timeline and changing the update wiring to use those tracks – by just changing the curve and saving it you should be able to quickly try out different actions.
- Try using different types of Spline Generation method and try to manually change the spline points to fine tune the movement around a spline. Experiment with the Rotation values from the Spline.
- Try to Combine multiple concepts into a single blueprint (e.g. a curve that both moves forwards and flips the rotation as well). The easiest way of doing this is adding multiple tracks to the same timeline, but when you do this, remember that all tracks will run for the same time and same loop options.
- You can also add multiple Timelines and set them to play from the start at the exact same time to also accomplish the above – this will allow you to
- Try to add a new Rotation Track to the spline example so that the rotation is independent of the curve path.
- Experiment with Transforming and Scaling the Curves and the Post Extrapolation values, such as Oscillate or Cycle with Offset etc.
- You can duplicate the Map as well to try out different layouts in separate maps – **this will allow you to prototype your different target types easily.**
- Try and build a layout setting different delays on each node so that you can create a set of targets that run for a certain amount of time.
- Research and try to apply the concept of the Colour Curve, Vector Curve and in particular the Event curve to your scenarios. Try and recreate the disappear and appear curve using an event curve – you can use this to show/hide and actor from the level.